

**ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA**
Universidad de Córdoba



TRABAJO DE FIN DE GRADO

Ecosistema de Domótica Local Impulsado por Agentes de IA

**AI-Driven Local Home
Automation Ecosystem**

Autor:
Jesús Fernández López

Titulación:
Grado en Ingeniería
Informática

Tutor:
Dr. Rafael Muñoz Salinas

Córdoba, 9 de junio de 2026



Declaración de Originalidad

D. **Jesús Fernández López**, con DNI 31876114T, estudiante del Grado en Ingeniería Informática de la Escuela Politécnica Superior de Córdoba, en relación con el Trabajo de Fin de Grado titulado «*Ecosistema de Domótica Local Impulsado por Agentes de IA*» declara que:

- El trabajo presentado es original y ha sido realizado íntegramente por el autor.
- No ha sido presentado anteriormente para la obtención de ningún otro título universitario.
- Se han citado debidamente todas las fuentes utilizadas, tanto bibliográficas como de software y hardware, siguiendo el formato de citación académico establecido.
- El uso de herramientas de inteligencia artificial durante el desarrollo se ha limitado a tareas de asistencia en la redacción y generación de fragmentos de código, siempre bajo supervisión directa del autor, tal como se detalla en el capítulo correspondiente de la memoria.

Y para que así conste, firmo la presente declaración en Córdoba, a 9 de junio de 2026.

Fdo.: Jesús Fernández López

Nota: Esta declaración se complementa con la firma electrónica del autor en los ficheros digitales entregados, de acuerdo con la normativa de la Universidad de Córdoba.

Agradecimientos

A mi tutor, el Dr. Rafael Muñoz Salinas, por su apoyo constante, por la flexibilidad que me ha dado en todo momento para explorar caminos que no siempre estaban en el guión, y por lo rápido que siempre ha respondido cuando lo he necesitado. Estoy muy agradecido por su cercanía, su confianza y por haberme acompañado durante todo este proceso.

A mi familia, por aguantar estoicamente que el dormitorio se convirtiera en un taller de electrónica durante meses, entre cables, routers y ventiladores desmontados.

A los desarrolladores de OpenClaw, Ollama, Qwen, Home Assistant, tinytuya, bleak y bge-m3, cuyo trabajo *open-source* ha hecho posible este proyecto. En particular, a la comunidad de ingeniería inversa que dedica su tiempo a descifrar protocolos cerrados para que otros puedan construir sobre ellos.

Resumen

Este Trabajo de Fin de Grado presenta el diseño, implementación y validación de Luxe Core AI, un sistema de monitorización y control de confort térmico para el hogar que opera de forma completamente autónoma en una red local aislada de Internet, sin depender de servicios en la nube para ninguna de sus funciones críticas. El sistema integra dispositivos heterogéneos, como sensores ambientales Xiaomi mediante BLE, un enchufe inteligente Tuya controlado localmente por protocolo 3.5 y un ventilador de techo FanLamp F8808 cuyo protocolo BLE propietario fue descubierto mediante ingeniería inversa, bajo una arquitectura de tres capas orquestada por inteligencia artificial en el *edge*.

El núcleo del sistema es un enrutador inteligente de tres niveles (Model Router v2) que clasifica y procesa comandos en lenguaje natural sin enviar datos a servidores externos. El primer nivel, un sistema de *pattern matching* con 462 patrones, captura aproximadamente el 92 % de los comandos diarios con latencia inferior a 1 milisegundo. Los niveles superiores emplean modelos de lenguaje locales (*qwen2.5-coder:1.5b* y *mistral:7b*) para comandos ambiguos o conversacionales. Se ha implementado un sistema de auto-recuperación basado en *circuit breakers*, reintentos con *backoff* exponencial y un *SelfHealManager* autónomo.

Los resultados demuestran que es posible construir un sistema de control ambiental 100 % local con hardware de consumo (servidor Ryzen 5, 16 GB RAM) y un presupuesto de 0 € en licencias y servicios cloud, manteniendo la privacidad de los datos del hogar como principio de diseño fundamental.

Palabras clave: Edge AI, control térmico local, IoT, modelos de lenguaje, privacidad, BLE, Tuya, enrutamiento inteligente, OpenClaw.

Abstract

This Bachelor's Thesis presents the design, implementation and validation of Luxe Core AI, a local thermal comfort monitoring and control system that operates completely autonomously on an Internet-isolated local network, with no dependency on cloud services for any of its critical functions. The system integrates heterogeneous devices, such as Xiaomi environmental sensors via BLE, a Tuya smart plug controlled locally through protocol 3.5 and a FanLamp F8808 ceiling fan whose proprietary BLE protocol was uncovered through reverse engineering, under a three-layer architecture orchestrated by edge artificial intelligence.

The system's core is a three-tier intelligent router (Model Router v2) that classifies and processes natural language commands without sending data to external servers. The first tier, a pattern-matching engine with 462 patterns, captures approximately 92 % of daily commands with sub-millisecond latency. Upper tiers employ local language models (qwen2.5-coder:1.5b and mistral:7b) for ambiguous or conversational commands. A self-healing system based on circuit breakers, exponential backoff retries and an autonomous SelfHealManager has been implemented.

The results show that it is feasible to build a 100 % local environmental control system using consumer-grade hardware (Ryzen 5 server, 16 GB RAM) with an integration budget of 0 € in software licences, keeping household data privacy as a fundamental design principle.

Keywords: Edge AI, local thermal control, IoT, language models, privacy, BLE, Tuya, intelligent routing, OpenClaw.

Índice general

Agradecimientos	3
Resumen	5
Abstract	7
Índice de figuras	VII
Índice de tablas	IX
1. Introducción	1
1.1. Motivación Personal	2
2. Definición del Problema	5
2.1. Dependencia de Conectividad	5
2.2. Exposición de Datos Sensibles	6
2.3. Latencia y Capacidad de Respuesta	6
2.4. Heterogeneidad de Protocolos	6
2.5. Restricción Económica como Factor de Diseño	7
2.6. Síntesis del Problema	7
3. Objetivos	9
3.1. Objetivos Específicos	9
3.2. Objetivos de Evaluación	10
3.3. Restricciones Asumidas	11
4. Antecedentes	13
4.1. Estado del Arte en Domótica	13
4.2. Plataformas de Automatización del Hogar	13
4.3. Edge AI y Modelos de Lenguaje Locales	14
4.3.1. TinyML y Procesamiento en el Edge	14
4.3.2. Asistentes de Voz Locales	14

4.4.	Comparativa de Modelos Locales	15
4.5.	Control Local de IoT	16
4.5.1.	Protocolos y Arquitecturas	16
4.5.2.	Firmwares Alternativos	16
4.5.3.	Bibliotecas de Control Directo	16
4.6.	Frameworks de Agentes	17
4.7.	Resumen del Contexto Tecnológico	18
5.	Restricciones y Metodología	19
5.1.	Restricciones del Proyecto	19
5.1.1.	Restricción Temporal	19
5.1.2.	Restricción Económica	19
5.2.	Decisiones Arquitectónicas	20
5.2.1.	Evolución de la Infraestructura	20
5.2.2.	Paradigma Políglota	20
5.2.3.	Evolución de la Selección de Modelos	21
5.2.4.	Aislamiento de Red: La Arquitectura Air-Gapped	22
5.2.4.1.	Hotspot Spoofing para Dispositivos Tuya	23
5.2.4.2.	Lectura de Sensores Xiaomi LYWSD03MMC	23
5.2.5.	Selección del Sistema Operativo y el Servidor	24
5.2.5.1.	Por qué Ubuntu Server 24.04 LTS	24
5.2.5.2.	Herramientas de Gestión	25
5.3.	Metodología de Trabajo	25
5.3.1.	Control de Versiones y Propiedad Intelectual	25
5.3.2.	Despliegue Inicial: Incidencias y Soluciones	25
5.3.2.1.	El Puerto LAN/WAN Híbrido	26
5.3.2.2.	Pasarela de Red de Contingencia	26
5.3.3.	Rigor Documental	27
5.3.4.	Uso de la Inteligencia Artificial en el Desarrollo	27
6.	Especificación de Requisitos	29
6.1.	Requisitos Funcionales	29
6.2.	Requisitos No Funcionales	30
7.	Recursos	33
7.1.	Hardware	33
7.1.1.	Presupuesto de Hardware	34
7.2.	Tabla Comparativa de Alternativas Consideradas	34
7.3.	Software y Librerías	35
7.4.	Dependencias de Red	36

8. Especificación del Sistema	37
8.1. Visión General de la Arquitectura	37
8.2. Topología de Red Local	39
8.2.1. Asignación de Direcciones IP	39
8.2.2. Ventana de Conectividad de Contingencia	40
8.2.3. Topología Física	40
8.3. Decisiones de Diseño	40
8.3.1. Modelos Evaluados para el Razonador (Tier 2)	40
8.4. Módulos del Servidor	42
8.4.1. Orquestador OpenClaw (Node.js)	42
8.4.2. Arquitectura de Enrutamiento Inteligente (Model Router v2)	42
8.4.3. Auto-Recuperación y Tolerancia a Fallos	44
8.4.4. Daemon de Sensorización Continua	45
8.4.5. Asesor de Confort Térmico (Comfort Advisor)	46
8.4.6. Asesor Inteligente con Contexto Exterior (Smart Advisor)	46
8.4.7. Comunicación y Notificaciones	48
8.4.8. Infraestructura de Despliegue	48
8.4.9. Métricas de Rendimiento	48
8.4.10. Canales de Usuario	49
8.4.11. Módulo de Integración Tuya	49
8.4.12. Módulo de Sensores BLE	50
8.4.13. Criterios de Integración y Restricciones	50
8.4.14. Impacto de la Restricción Económica en las Decisiones de Implementación	50
8.5. Firmware del Nodo ESP32 (firmware/)	51
8.6. Integraciones de Hardware	52
8.6.1. Dispositivos Tuya	52
8.6.2. Sensores Xiaomi LYWSD03MMC	52
8.6.3. Ventilador de Techo FanLamp Pro F8808	52
8.6.3.1. Problema de Integración	52
8.6.3.2. Descubrimiento del Protocolo	52
8.6.3.3. Control desde el Ordenador	53
8.6.3.4. Evolución del Controlador: de BLE directo a Puente ESP32	53
8.7. Estructura del Repositorio	54
9. Validación y Pruebas	61
9.1. Integración Tuya	61
9.2. Sensores BLE Xiaomi	61
9.3. Ventilador FanLamp F8808	62
9.4. Sensor Daemon	62

9.5. Model Router v2	63
9.6. Prueba de Cobertura del Tier 0	63
9.7. Tabla de Latencia Comparativa	63
9.8. Circuit Breaker y Tolerancia a Fallos	64
9.9. Suite de Integración Automatizada	64
10. Conclusiones	67
10.1. Logros Alcanzados	67
10.2. Estado Actual del Desarrollo	69
10.3. Líneas de Trabajo Futuras	69
10.4. Reflexión Personal	71
11. Bibliografía	73
Anexos	77
A. Manual de Usuario	79
A.1. Introducción	79
A.2. Requisitos del Sistema	79
A.2.1. Hardware Necesario	79
A.2.2. Red Necesaria	80
A.3. Instalación Paso a Paso	80
A.3.1. Conexión Física	80
A.3.2. Instalación de Software y Primer Arranque	81
A.3.3. Primer Comando	83
A.4. Canales de Comunicación	83
A.5. Comandos Disponibles	84
A.5.1. Iluminación	84
A.5.2. Ventilador	84
A.5.3. Enchufe Inteligente	84
A.5.4. Escenas Predefinidas	85
A.5.5. Consultas	85
A.5.6. Tabla Resumen de Comandos	85
A.6. Notificaciones Proactivas	85
A.7. Seguridad	86
A.8. Solución de Problemas Comunes	86
A.8.1. Indicadores LED del ESP32	86
A.8.2. Casos Frecuentes	87
A.9. Añadir un Dispositivo Nuevo	88
A.9.1. Añadir un Dispositivo Tuya	88

A.9.2. Añadir un Sensor BLE Xiaomi	88
A.10. Mantenimiento	89
A.10.1. Actualizaciones de Software	89
A.10.2. Sustitución de Pilas	89
A.10.3. Backup de Configuración	89
A.10.4. Restauración de Configuración	89
A.10.5. Restauración del ESP32	90
A.10.6. Diagnóstico con OpenClaw	90
A.11. Preguntas Frecuentes (FAQ)	90
A.12. Checklist de Puesta en Marcha	91
A.12.1. Fase 1: Conexión Física	91
A.12.2. Fase 2: Instalación de Software Base	92
A.12.3. Fase 3: Configuración de OpenClaw	92
A.12.4. Fase 4: Verificación del Sistema	92
B. Manual de Código	95
B.1. Enrutador de Tres Niveles (Model Router v2)	95
B.2. Sistema de Zero-Inference (Tier 0)	96
B.3. Daemon de Sensorización Continua	96
B.4. Asesor de Confort Térmico (Comfort Advisor)	97
B.5. Control del Ventilador FanLamp (Anuncios BLE)	97
B.6. Integración Tuya (Protocolo Local 3.5)	98
B.7. Lectura GATT de Sensores Xiaomi	98
B.8. Auto-Recuperación (Circuit Breaker)	99
C. Glosario de Acrónimos	101

Índice de figuras

7.1. Componentes hardware del subsistema HVAC: ESP32, regulador de voltaje y caja de protección.	35
8.1. Arquitectura de tres capas. Las flechas indican flujo de comunicación entre componentes.	38
8.2. Topología de despliegue sobre la red air-gapped. Las líneas continuas indican Ethernet, las discontinuas Wi-Fi, y las punteadas BLE. El control del ventilador y el sensor se realiza a través del ESP32 como puente BLE, mientras que el enchufe Tuya se comunica directamente por IP local. . .	39
8.3. Flujo de enrutamiento del Model Router v2. El mensaje pasa por tres niveles de complejidad creciente, con un circuit breaker que deriva al Tier 2 si el clasificador 1.5B no está disponible.	43
8.4. Zonas de confort térmico adaptadas al clima andaluz. Las zonas se asignan en función del índice de calor NOAA, combinando temperatura y humedad relativa.	46
8.5. Interacción por Telegram.	56
8.6. <i>Dashboard</i> web de OpenClaw.	57
8.7. Enchufe Tuya y humidificador.	57
8.8. Sensor Xiaomi LYWSD03MMC.	58
8.9. Decodificación de los 5 bytes de la característica <code>ebe0ccc1</code> del sensor Xiaomi LYWSD03MMC. El payload se lee por GATT sin autenticación. .	58
8.10. Ventilador FanLamp F8808 instalado.	59
9.1. Comparación de latencia antes y después de la implementación del Model Router v2.	65
9.2. Cobertura del sistema de zero-inference (Tier 0) sobre corpus de 500 comandos reales.	66

Índice de tablas

5.1.	Comparativa de latencias y fiabilidad de los modelos evaluados para el Tier 2. Las pruebas se realizaron sobre el corpus de 500 comandos reales del proyecto en un Ryzen 5 con 16 GB RAM.	22
7.1.	Desglose de costes de hardware del proyecto Luxe Core AI.	34
7.2.	Estimación de costes anuales de un ecosistema domótico equivalente basado en servicios cloud (excluyendo hardware, común en ambos casos). Los precios son orientativos y varían según proveedor, volumen de uso y región.	35
7.3.	Comparativa de alternativas tecnológicas consideradas para cada componente del sistema.	36
8.1.	Asignación estática de direcciones IP en el segmento air-gapped.	40
8.2.	Architecture Decision Records (ADR) del proyecto, con referencias a los capítulos donde se detalla cada decisión.	41
8.3.	Modelos evaluados como candidatos para el Tier 2 de razonamiento. Las mediciones se realizaron sobre el corpus de 500 comandos reales del proyecto.	41
8.4.	Mejora de latencia tras la implementación del Model Router v2 y el Sensor Daemon.	49
8.5.	Comandos disponibles para el ventilador FanLamp F8808.	53
8.6.	Comparativa entre las dos fases del controlador del ventilador FanLamp.	55
9.1.	Mejora de latencia tras la implementación del Model Router v2 y el Sensor Daemon.	64
10.1.	Estado de los módulos del proyecto a la fecha de esta memoria.	69
A.1.	Resumen de comandos disponibles por categoría.	85

Introducción

El presente proyecto aborda el diseño, implementación y validación de un sistema de monitorización y control de confort térmico para el hogar que opera de forma completamente autónoma en una red local privada, eliminando cualquier dependencia de servicios en la nube para su funcionamiento. El sistema integra un enchufe inteligente, un sensor de temperatura y humedad y un ventilador de techo, todos ellos controlados mediante inteligencia artificial local que interpreta comandos en lenguaje natural y toma decisiones de confort sin enviar datos al exterior.

Este trabajo nace de una observación sencilla: los hogares inteligentes de hoy no son tan inteligentes como parecen. La mayoría de los dispositivos domóticos dependen de servidores externos para funcionar, de modo que cuando falla la conexión a Internet, o cuando el fabricante decide dejar de dar soporte, el usuario se queda sin poder encender una luz o regular el termostato.

El problema no es menor. Cada vez que un sensor de temperatura envía datos a la nube, o que un asistente de voz procesa una orden en servidores remotos, la privacidad del usuario queda expuesta. Los patrones de comportamiento doméstico (a qué hora se levanta, qué temperatura prefiere para dormir, qué días está en casa) viajan a servidores de terceros, a menudo sin que el usuario sea consciente ni haya dado un consentimiento informado.

El proyecto que se presenta en esta memoria, **Luxe Core AI**, implementa esta idea: en lugar de delegar el procesamiento a la nube, los modelos de lenguaje se ejecutan en un servidor doméstico, y los datos de los sensores nunca abandonan la red local.

Esta decisión de diseño tiene consecuencias prácticas. La latencia de respuesta se reduce de segundos a milisegundos para la mayoría de los comandos. El sistema sigue funcionando aunque el router pierda la conexión con el exterior. Y, sobre todo, los datos del hogar pertenecen exclusivamente a quien vive en él.

Una particularidad del proyecto es que integra dispositivos que no fueron diseñados

para interoperar. El proyecto incluye un enchufe inteligente Tuya controlado sin pasar por la nube del fabricante (que a su vez alimenta un humidificador ultrasónico de 24 V, un dispositivo originalmente “tonto” que solo necesita corriente y que, al conectarlo al enchufe, adquiere control remoto), un sensor de temperatura Xiaomi leído mediante Bluetooth Low Energy sin necesidad de su aplicación oficial, y un ventilador de techo de 35 € adquirido en AliExpress cuyo protocolo de comunicación fue descubierto mediante ingeniería inversa. Todos estos dispositivos, que individualmente son “tontos”, adquieren capacidad de razonamiento al ser orquestados por la capa de inteligencia artificial. El resultado es que la interoperabilidad no depende de estándares cerrados ni de la buena voluntad de los fabricantes, sino de una capa de software que entiende y coordina hardware heterogéneo.

El desarrollo ha seguido dos fases claramente diferenciadas. Durante la primera, se utilizó la infraestructura de la Universidad de Córdoba (servidores con GPU RTX accesibles por VPN) para prototipar la arquitectura de agentes y validar que el enfoque era viable. En la segunda fase, todo el sistema migró a una estación de trabajo doméstica, eliminando cualquier dependencia externa. Esta evolución no fue un retroceso, sino el paso natural hacia el objetivo central del proyecto: comprobar que el control del entorno doméstico mediante IA puede funcionar sin depender de infraestructuras externas.

1.1. Motivación Personal

Más allá de las consideraciones técnicas, este proyecto responde a una motivación personal: aprender a trabajar con tecnologías de *Edge AI* en un entorno real, con hardware real, y comprobar por mí mismo hasta dónde pueden llegar los modelos de lenguaje pequeños ejecutados en local.

Cuando empecé el TFG, los modelos de lenguaje grandes (más de 30 B parámetros) copaban todos los titulares, pero requerían GPUs de varios miles de euros que estaban fuera de mi alcance. En paralelo, modelos mucho más ligeros (como `qwen2.5-coder:1.5b` o `bge-m3`) estaban demostrando que se podía hacer mucho con muy poco: clasificación de comandos, análisis semántico, generación de texto coherente, todo en menos de 500 MB de RAM. Quería comprobar si esos modelos eran suficientemente buenos para un caso de uso real, con las limitaciones de un servidor doméstico. El proceso de evaluación, detallado en la Sección 5.2.3, condujo a una arquitectura mixta: Qwen2.5 1.5B como clasificador determinista y Mistral 7B como razonador conversacional, ambos ejecutándose en CPU con cuantización optimizada.

Los resultados han superado mis expectativas. El clasificador 1.5B responde en 300–500 ms con JSON determinista, el razonador Mistral 7B conversa con naturalidad en 27–60 s, y el sistema completo funciona 24/7 sin intervención. Esta experiencia me ha confirmado que los modelos pequeños y locales son una opción viable para aplicaciones con recursos limitados, y que su capacidad sigue mejorando.

A nivel de documentación, esta memoria se ha redactado con $\text{\LaTeX}$ y se gestiona con el mismo control de versiones (Git) que el código del proyecto. Esto permite mantener la trazabilidad entre las decisiones de diseño, el código que las implementa y la documentación que las describe.

Luxe Core AI es, en definitiva, un caso práctico de cómo la inteligencia artificial puede aplicarse al entorno doméstico sin renunciar a la privacidad ni depender de infraestructuras externas.

Cabe señalar que la estructura de esta memoria no sigue de forma literal el orden de la guía de desarrollo oficial del GII [1], pues se ha optado por una organización temática que facilita la lectura del proceso completo. No obstante, toda la información exigida por el reglamento (introducción, objetivos, antecedentes, análisis del problema, diseño, conclusiones, bibliografía y manuales) está presente y es fácilmente localizable a lo largo de los capítulos y anexos.

Definición del Problema

Los sistemas de domótica más extendidos hoy en día comparten un mismo modelo de funcionamiento: el hogar se conecta a la nube, y la inteligencia que lo gobierna reside en los servidores del fabricante. Amazon Alexa, Google Home, Tuya Smart, por citar solo los más conocidos, ejecutan el procesamiento de órdenes y la toma de decisiones en centros de datos remotos, dejando al dispositivo local la mera función de terminal de entrada y salida.

Este modelo, que ha democratizado el acceso a la domótica, acarrea problemas estructurales difíciles de ignorar.

2.1. Dependencia de Conectividad

El primero es la dependencia de una conexión permanente a Internet. Si el router se queda sin red, por una avería del operador, un corte programado o simplemente porque el usuario vive en una zona con cobertura deficiente, el hogar inteligente se convierte en un conjunto de objetos inertes. El usuario no puede encender una luz, regular el termostato ni consultar la temperatura de su casa. Esta dependencia es particularmente grave en hogares donde la domótica cubre funciones esenciales, como el control de la calefacción o la ventilación.

Estudios del sector [2] muestran que una parte relevante de los hogares españoles carece de acceso a Internet de banda ancha, con diferencias significativas entre zonas urbanas y rurales. A esto se suman los inevitables cortes de suministro eléctrico, eventualidades imprevisibles que afectan a cualquier vivienda. Durante esos cortes no solo se pierde la conectividad externa, sino que los routers domésticos (que dependen de la corriente) dejan de funcionar por completo, lo que paraliza cualquier sistema domótico que dependa de servidores externos para procesar sus rutinas locales.

2.2. Exposición de Datos Sensibles

El segundo problema es la exposición de datos personales. Cada interacción con un asistente de voz, cada cambio de temperatura registrado por un sensor, cada horario de encendido y apagado de luces, viaja a servidores gestionados por terceros. El usuario no tiene control sobre dónde se almacenan esos datos, durante cuánto tiempo se conservan ni con qué fines se utilizan.

Casos sonados no faltan. En 2019, una investigación de Bloomberg reveló que empleados de Amazon escuchaban grabaciones de Alexa para mejorar el reconocimiento de voz, sin que los usuarios fueran informados [3]. Más allá de incidentes concretos, el problema es estructural: el modelo de negocio de los fabricantes se basa en la recolección de datos del usuario, que quedan almacenados en servidores de terceros sin que este tenga control sobre su uso o conservación.

El problema de fondo no es técnico sino de incentivos: mientras la domótica comercial dependa de servidores externos para funcionar, la privacidad del usuario será un elemento negociable en la ecuación económica del fabricante.

2.3. Latencia y Capacidad de Respuesta

El tercer problema es la latencia. Incluso en conexiones de banda ancha, el trayecto de ida y vuelta hasta la nube introduce retardos perceptibles. Una orden tan simple como “apaga la luz” puede tardar varios segundos en ejecutarse, lo que resulta frustrante en el uso diario. En aplicaciones que requieren respuesta inmediata, como el control de un ventilador cuando la temperatura sube por encima de un umbral, esos segundos marcan la diferencia.

La revisión sistemática de herramientas y aplicaciones de TinyML realizada por Kallimani et al. [4] documenta cómo el procesamiento local puede reducir la latencia de extremo a extremo en órdenes de magnitud, pasando de segundos a milisegundos para tareas de clasificación y control. Esta reducción no solo beneficia la experiencia del usuario, sino que también tiene un impacto energético positivo: un sistema que responde en milisegundos puede reaccionar antes a cambios ambientales, reduciendo el tiempo de funcionamiento innecesario de los actuadores.

2.4. Heterogeneidad de Protocolos

A los problemas anteriores se suma la fragmentación del ecosistema IoT. Los estándares de comunicación son múltiples y raramente interoperan de forma nativa: Tuya, Zigbee, Z-Wave, Bluetooth Low Energy, bus LIN para climatización [5]. Cada fabricante apuesta por su propio protocolo, y el usuario que quiere integrar dispositivos de distintas marcas se encuentra con que no hablan entre sí. Las plataformas comerciales resuelven

esto llevándolo todo a la nube, pero eso reintroduce los problemas de dependencia y privacidad.

Esta heterogeneidad no es un accidente: es una estrategia comercial. Los ecosistemas cerrados crean barreras de salida para el usuario, que no puede mezclar dispositivos de distintos fabricantes sin perder funcionalidades [6].

2.5. Restricción Económica como Factor de Diseño

Por último, el proyecto opera bajo una restricción económica real. El autor es estudiante sin ingresos, y cada dispositivo ha sido seleccionado priorizando el coste mínimo frente a la comodidad de integración. Un analizador de protocolos BLE (como el nRF52840, con un coste superior a 50 €) o un mando universal por infrarrojos habrían acelerado el desarrollo, pero se ha optado por la ingeniería inversa con herramientas ya disponibles.

Esta restricción no es una limitación del proyecto, sino una decisión deliberada: demostrar que la domótica inteligente puede construirse con dispositivos de consumo masivo y presupuestos ajustados. El coste total en licencias de software y servicios cloud ha sido de 0 €, gracias a que todos los dispositivos o bien ya estaban en posesión del autor, o bien fueron adquiridos con anterioridad por motivos ajenos al proyecto.

2.6. Síntesis del Problema

Todos estos problemas (dependencia de red, exposición de datos, latencia, fragmentación de protocolos y restricción económica) comparten una misma raíz: la delegación de la inteligencia del hogar a infraestructuras externas. Este proyecto propone una respuesta directa: una configuración de control local que integra hardware heterogéneo mediante software desacoplado y orquestado por inteligencia artificial local, sin depender de la nube para ninguna función crítica.

A partir de este diagnóstico, el capítulo siguiente define los objetivos concretos que se persiguen y los indicadores que permitirán medir su cumplimiento.

Objetivos

El objetivo general del presente Trabajo de Fin de Grado es el diseño, implementación y validación de un sistema de monitorización y control de confort térmico para el hogar que opere de forma autónoma en una red local privada, eliminando cualquier dependencia de servicios en la nube para su funcionamiento.

3.1. Objetivos Específicos

A continuación se desglosa cada objetivo específico en subobjetivos medibles que permitan verificar su cumplimiento:

1. Diseñar una arquitectura de microservicios local.

- Definir las interfaces de comunicación entre cada componente del sistema (orquestador, módulo HVAC, módulo IoT, sensores), garantizando que sean independientes y sustituibles.
- Implementar el orquestador OpenClaw como servicio systemd con auto-arranque y reinicio automático.
- Documentar la arquitectura mediante diagramas de despliegue y de flujo de datos.
- Verificar que ningún componente requiere acceso a Internet para su funcionamiento básico.

2. Implementar el control de la unidad HVAC por bus LIN.

- Diseñar el firmware en C++20 para el ESP32 DevKit V1 que actúe como puente entre la red local (serie/USB) y el bus LIN de la máquina General/Fujitsu.

- Implementar el parser de tramas LIN basado en la documentación del proyecto `esphome-fujitsu-halcyon` [7].
- Validar la comunicación bidireccional: lectura de estado del termostato y envío de comandos de control.
- Garantizar que el firmware mantenga el último estado válido en caso de pérdida de comunicación con el servidor (RNF-04).

3. Integrar dispositivos IoT heterogéneos.

- Lectura de sensores Xiaomi LYWSD03MMC mediante BLE GATT directo, utilizando exclusivamente la librería `bleak` [8], sin depender de la nube de Xiaomi ni de firmware alternativo (RF-02).
- Control local del enchufe inteligente Tuya mediante `tinytuya` [9] sobre la red air-gapped, sin pasar por los servidores de Tuya (RF-03). Tiempo de respuesta inferior a 2 s.
- Ingeniería inversa completa del protocolo BLE del ventilador FanLamp F8808, incluyendo el mapeo de todos los comandos funcionales (RF-04).
- Documentar cada integración con los pasos necesarios para reproducirla en otro entorno.

Todas las integraciones deben cumplir los siguientes criterios transversales:

- **Coste cero en licencias o suscripciones:** no se utilizarán servicios de pago, APIs externas ni plataformas que requieran cuota.
- **Reproducibilidad:** cualquier persona con el mismo hardware debe poder replicar la integración siguiendo la documentación.
- **Validación sobre hardware real:** todas las pruebas deben realizarse sobre los dispositivos físicos en la red air-gapped, no en simulación.

3.2. Objetivos de Evaluación

Para medir el grado de cumplimiento del proyecto, se establecen los siguientes indicadores:

1. **Cobertura del enrutador de comandos:** el sistema debe ser capaz de interpretar y responder correctamente al menos el 90 % de los comandos en lenguaje natural que recibe, sin requerir reformulación por parte del usuario.
2. **Latencia de respuesta:** la mediana de latencia para comandos directos (encender/apagar, cambio de velocidad) debe ser inferior a 2 s en la red local.

3. **Disponibilidad del sistema:** los servicios críticos (orquestador, sensor daemon, router de modelos) deben mantener una disponibilidad superior al 99 % en condiciones normales de operación.
4. **Aislamiento de red:** debe verificarse que ningún paquete de datos del sistema abandone la subred 192.168.1.0/24 durante su funcionamiento normal.

3.3. Restricciones Asumidas

El proyecto asume las siguientes restricciones que acotan el alcance de los objetivos:

- El desarrollo del firmware HVAC (ESP32 + LIN) queda como objetivo principal pero con posible desplazamiento a trabajo futuro si las pruebas de integración no se completan dentro del calendario académico.
- No se implementará una aplicación móvil nativa; el control se realiza exclusivamente mediante Telegram y el *dashboard* web de OpenClaw.
- La base de datos SQLite de analítica energética se considera un objetivo secundario, priorizando la funcionalidad de control en tiempo real.

Con los objetivos definidos, el capítulo siguiente examina el estado del arte y las tecnologías disponibles para abordarlos.

Antecedentes

4.1. Estado del Arte en Domótica

La domótica ha experimentado en la última década una rápida evolución, impulsada principalmente por la proliferación de dispositivos conectados de bajo coste. Plataformas como Amazon Alexa, Google Home o Apple HomeKit han democratizado el acceso a la automatización del hogar, pero lo han hecho a costa de crear ecosistemas cerrados y dependientes de la conectividad a Internet y de la voluntad comercial de sus fabricantes.

Como alternativa *open-source*, proyectos como **Home Assistant** [6] han ganado una considerable tracción en la comunidad, demostrando que es viable gestionar múltiples protocolos IoT desde un servidor local. Home Assistant soporta más de 2000 integraciones y se ejecuta en hardware modesto como una Raspberry Pi. Sin embargo, aunque dispone de addons que permiten conectarse a APIs de inteligencia artificial (OpenAI, Google), estas funcionalidades requieren necesariamente acceso a servicios externos, reintroduciendo la dependencia de nube que este proyecto busca eliminar.

4.2. Plataformas de Automatización del Hogar

Junto a Home Assistant, existen otras plataformas de automatización consolidadas. **OpenHAB** [10] ofrece un enfoque más modular y empresarial, basado en Java, con un motor de reglas potente pero una curva de aprendizaje más pronunciada. **ESPHome** [11], por su parte, está orientado a flashear firmware en microcontroladores ESP32/ESP8266 y se integra estrechamente con Home Assistant, pero su ámbito es la programación de dispositivos, no la orquestación de alto nivel.

Ninguna de estas plataformas incorpora de forma nativa un modelo de lenguaje local para interpretar comandos en lenguaje natural o tomar decisiones contextuales. Las

automatizaciones se definen mediante reglas YAML (Home Assistant), scripts (OpenHAB) o configuración declarativa (ESPHome). Introducir esta capacidad requeriría, en todos los casos, añadir un componente externo que se comunicase con una API cloud o con un servidor de inferencia local como Ollama, incrementando la complejidad arquitectónica.

Para este proyecto, el enfoque fue el inverso: construir la orquestación inteligente como núcleo (OpenClaw + Model Router) y conectar los dispositivos mediante *drivers* Python ligeros, en lugar de partir de una plataforma de automatización generalista y añadirle IA como extensión. Esto evita depender de ecosistemas diseñados para la nube y mantiene la latencia de control por debajo de los 2 s.

4.3. Edge AI y Modelos de Lenguaje Locales

La madurez alcanzada por herramientas como **Ollama** [12] ha sido el catalizador tecnológico que hace viable la fase actual del proyecto. Ollama permite ejecutar modelos de lenguaje de tamaño medio (7B–14B parámetros) en hardware de consumo con rendimiento aceptable. Esto elimina la dependencia de APIs externas de pago como OpenAI o Google Gemini para la lógica cognitiva del sistema, haciendo que el control local de los datos sea completamente realizable.

El panorama de modelos pequeños ha evolucionado rápidamente. La familia **Qwen2.5** [13], de Alibaba Cloud, demostró que modelos con 1.5B y 7B parámetros pueden competir en tareas específicas con modelos mucho mayores. En febrero de 2026, Alibaba lanzó **Qwen3.5** [14], una evolución con mejoras en contexto (256K tokens), multilingüismo (201 idiomas) y modo *thinking* nativo. Sin embargo, como se detalla en el Capítulo 5, las pruebas en el hardware del proyecto mostraron que el *thinking* forzado de Qwen3.5 lo hacía inviable en CPU para aplicaciones con requisitos de latencia, por lo que fue evaluado pero finalmente descartado.

4.3.1. TinyML y Procesamiento en el Edge

El campo del TinyML [4] ha demostrado que es posible ejecutar modelos de *machine learning* en microcontroladores con apenas kilobytes de memoria. Aunque este proyecto no llega a ese extremo (los modelos se ejecutan en un servidor x86), los principios del TinyML (minimización de dependencias, optimización para hardware limitado, procesamiento local de datos) informan el diseño del sistema.

4.3.2. Asistentes de Voz Locales

Actualmente es posible construir sistemas de reconocimiento y síntesis de voz que funcionen completamente desconectados de Internet. Aunque la calidad sigue siendo inferior a la de los asistentes comerciales (que pueden aprovechar modelos con cientos de miles de millones de parámetros), la brecha se reduce año tras año. La decisión de

este proyecto de no incorporar entrada por voz de momento responde a esta limitación objetiva.

4.4. Comparativa de Modelos Locales

Para elegir los modelos que componen el enrutador de tres niveles (Tier 1 como clasificador, Tier 2 como razonador), se evaluaron varias alternativas disponibles en Ollama, teniendo en cuenta las limitaciones del hardware (Ryzen 5, 16 GB RAM, sin GPU dedicada). Los criterios fueron: consumo de RAM, capacidad de respuesta en español, consistencia en la generación de JSON estructurado y velocidad de inferencia en CPU.

- **Qwen2.5 1.5B** [13]: Ocupa aproximadamente 986 MB en RAM y responde en 300–500 ms en CPU. Su generación de JSON estructurado es determinista incluso con `temperature=0.0`, cualidad esencial para el *parser* del enrutador. Se seleccionó como clasificador ligero (Tier 1).
- **Mistral 7B (Instruct v0.3)** [15]: Seleccionado como razonador (Tier 2) tras evaluar alternativas. Con cuantización Q4_K_M ocupa 4.4 GB en RAM y ofrece respuesta directa sin *thinking* forzado, con latencias de 27–60 s en CPU. Su calidad conversacional evita la sensación de estar interactuando con un sistema puramente determinista.
- **Qwen3.5 (2B, 4B, 9B)** [14]: Evaluado como posible sustituto de la familia Qwen2.5. Aunque ofrece ventajas en contexto (256K tokens) y multilingüismo (201 idiomas), su modo *thinking* forzado —no desactivable vía API— produce latencias de 3 a 12 minutos en CPU para el modelo de 9B, y mostró inestabilidad en la generación de JSON (12–18 % de fallos). Descartado para este proyecto por incompatibilidad con los requisitos de latencia (RNF-03).
- **Llama 3.2 (1B, 3B, 8B)** [16]: Su documentación oficial indica que está entrenado principalmente con datos en inglés. Ocupa 8 GB en RAM en su versión 8B, superando el margen disponible tras el clasificador. Las versiones 1B y 3B están optimizadas para dispositivos móviles y no mantienen la coherencia conversacional necesaria para el razonador. No se seleccionó.
- **Phi-3 (Microsoft)** [17]: Según su documentación, está entrenado con datos sintéticos y contenido educativo principalmente en inglés. Su rendimiento en español coloquial no está contemplado en las evaluaciones publicadas. No se seleccionó.

Para la generación de *embeddings* semánticos se seleccionó `bge-m3` [18] desde el inicio, por su eficiencia (apenas 500 MB en RAM), su capacidad multilingüe (español e inglés) y su madurez en el ecosistema Ollama.

4.5. Control Local de IoT

El ecosistema IoT actual ofrece múltiples aproximaciones para el control local de dispositivos, cada una con sus ventajas y limitaciones para un proyecto que opera sobre una red aislada.

4.5.1. Protocolos y Arquitecturas

El enfoque más extendido en domótica local es el uso de **MQTT** como bus de mensajería, con un *broker* (Mosquitto [19]) que centraliza la comunicación entre dispositivos y la lógica de control. Sin embargo, para integrar hardware comercial cerrado como el enchufe Tuya o el ventilador FanLamp, MQTT por sí solo no es suficiente: se necesitarían puentes adicionales (ESPHome [11], OpenMQTTGateway [20]) que traduzcan entre el protocolo propietario del dispositivo y el bus MQTT, añadiendo complejidad y puntos de fallo.

4.5.2. Firmwares Alternativos

Proyectos como **Tasmota** [21] y **ESPHome** [11] permiten flashear firmware libre en dispositivos basados en ESP8266/ESP32, eliminando la dependencia del software original del fabricante. En el caso del enchufe Tuya, la *Local Key* ya permite el control completo mediante el protocolo 3.5, haciendo innecesario el flasheo. Para el ventilador FanLamp, no se ha podido determinar el modelo exacto de chip BLE, por lo que un flasheo de firmware no es una opción viable sin un análisis más profundo del hardware.

OpenMQTTGateway [20] es un firmware para ESP32 que actúa como puente entre dispositivos BLE y un *broker* MQTT. Se consideró como posible intermediario para la lectura del sensor Xiaomi, pero se descartó por la sobrecarga de latencia que introduce frente a la alternativa de lectura GATT directa.

4.5.3. Bibliotecas de Control Directo

Para este proyecto se optó por un enfoque más simple: utilizar bibliotecas Python que se comunican directamente con cada dispositivo, sin intermediarios:

- **tinytuya** [9]: Permite el control local de dispositivos Tuya mediante el protocolo 3.5 sobre la red IP local, sin pasar por la nube del fabricante. La *Local Key* se extrae una única vez durante el emparejamiento inicial.
- **bleak** [8]: Librería BLE asíncrona que permite la lectura GATT directa de sensores como el Xiaomi LYWSD03MMC, sin necesidad de *broker* MQTT ni firmware intermedio.

Esta aproximación reduce la latencia (una sola conexión directa frente a una cadena de saltos), elimina puntos de fallo intermedios y simplifica el mantenimiento al tener menos componentes. La desventaja es que cada nuevo tipo de dispositivo requiere un *driver* específico, pero al tratarse de un entorno con un número reducido de dispositivos, la carga de desarrollo es asumible.

4.6. Frameworks de Agentes

El orquestador principal del proyecto es **OpenClaw** [22], un *framework* agéntico para Node.js que proporciona canales de usuario (Telegram, web), ejecución de herramientas y coordinación de modelos de lenguaje. Se eligió frente a otras opciones por su ligereza, simplicidad de configuración y modularidad.

Durante la fase de investigación se analizaron varias alternativas:

- **Hermes** [23]: Plataforma emergente con un mecanismo de autoaprendizaje que permite al sistema ajustar su comportamiento basándose en la retroalimentación del usuario. Su ecosistema es menos maduro que el de OpenClaw y su integración con modelos locales requería adaptación adicional. La idea del autoaprendizaje se adoptó como línea de desarrollo propia, implementada en el Smart Advisor.
- **AutoGPT** [24]: Diseñado para tareas autónomas de larga duración, con generación y ejecución recursiva de subobjetivos. Su sobrecarga computacional y su orientación a tareas abiertas lo hacen inadecuado para un sistema de control domótico que necesita respuestas en milisegundos.
- **LangChain** [25]: *Framework* para encadenar llamadas a modelos de lenguaje, más orientado a desarrollo de aplicaciones de IA que a la operación de un sistema de control en tiempo real. No proporciona canales de usuario ni gestión de herramientas de forma nativa.
- **n8n** [26]: Plataforma de automatización de flujos de trabajo basada en nodos. Potente para integraciones entre servicios, pero no está diseñada para gestionar interacciones conversacionales con modelos de lenguaje ni para ejecutar decisiones contextuales en tiempo real.

OpenClaw se mantuvo como opción principal por su madurez, su soporte nativo de canales (Telegram y *dashboard* web) y su capacidad para ejecutar herramientas Python mediante `exec`, que es precisamente el mecanismo utilizado para invocar los *drivers* de dispositivos.

4.7. Resumen del Contexto Tecnológico

En síntesis, el momento tecnológico actual ofrece las piezas necesarias para construir control local con IA en el *edge*: modelos de lenguaje pequeños pero capaces (Qwen2.5 [13], Mistral [15]), herramientas de inferencia local maduras (Ollama), *frameworks* agénticos ligeros (OpenClaw), librerías de integración IoT que funcionan sin nube (tinytuya, bleak) y una comunidad activa de ingeniería inversa que permite comprender dispositivos cerrados. Este proyecto integra estas piezas en un sistema que funciona 100 % en local, sin dependencias externas.

Restricciones y Metodología

El desarrollo de Luxe Core AI como TFG impone un marco de trabajo delimitado por restricciones técnicas, temporales y económicas que han moldeado las decisiones de diseño. Este capítulo las detalla y explica los *trade-offs* asumidos.

5.1. Restricciones del Proyecto

5.1.1. Restricción Temporal

El límite de un curso académico es la restricción más inflexible del proyecto. Define de forma estricta el alcance del producto mínimo viable (MVP). La metodología adoptada prioriza la iteración rápida y las funcionalidades troncales operativas frente a la sobreingeniería: es mejor tener un sistema que funcione con pocos dispositivos que uno diseñado para mil que nunca llegue a ejecutarse.

5.1.2. Restricción Económica

El proyecto ha operado con un presupuesto de hardware deliberadamente reducido. El autor es estudiante sin ingresos, lo que ha llevado a seleccionar los dispositivos más económicos disponibles (como el ventilador FanLamp F8808, el modelo más barato de AliExpress con conectividad) y a reutilizar hardware existente (el ESP32 DevKit V1, originalmente adquirido para el puente HVAC, se empleó también como radio BLE programable para la ingeniería inversa del ventilador).

Se ha descartado cualquier compra de hardware auxiliar no estrictamente imprescindible: nada de dongles analizadores BLE, mandos universales o *sniffers* de protocolos. Esta restricción no es una limitación del proyecto, sino una característica deliberada. Obliga a desarrollar competencias de ingeniería inversa que son transferibles a cualquier

dispositivo IoT futuro, y muestra que el control del entorno doméstico con IA no tiene por qué requerir grandes inversiones.

5.2. Decisiones Arquitectónicas

5.2.1. Evolución de la Infraestructura

El desarrollo de modelos de lenguaje locales requiere capacidad de cómputo que un portátil de estudiante no siempre puede proporcionar. Durante la primera fase del proyecto, se utilizó la infraestructura de la Universidad de Córdoba (servidores con GPU RTX, accesibles por VPN y SSH) para prototipar la arquitectura de agentes. Esta fase fue útil para validar el enfoque, pero dependía de una conexión externa y de recursos que el autor no controlaba.

En la segunda fase, el sistema migró a un servidor doméstico dedicado (torre Ryzen APU, 16 GB RAM). Esta migración no fue un retroceso, sino la materialización del objetivo central del proyecto: eliminar cualquier dependencia externa. Hoy, el sistema funciona al 100 % en local, con modelos de lenguaje ejecutándose en Ollama sobre el propio servidor, sin necesidad de GPU dedicada ni de conexión a Internet.

Durante esta fase también se evaluaron plataformas *open-source* de IoT como ESPHome, Tasmota y OpenMQTTGateway como posible base para las integraciones, pero se optó por un enfoque más ligero basado en *drivers* Python específicos para cada dispositivo, priorizando la simplicidad y el control sobre el hardware.

5.2.2. Paradigma Políglota

La naturaleza dual de la domótica con IA (tiempo real para el control hardware, flexibilidad cognitiva para la toma de decisiones) hace inviable un sistema monolítico en un solo lenguaje. Se ha adoptado una arquitectura de tres capas con lenguajes especializados:

- **Python 3.12** para los *drivers* de dispositivos (Tuya, BLE, FanLamp) y el enrutador de modelos. Python ofrece un ecosistema maduro de librerías (*tinytuya* [9], *bleak* [8]) y facilidad de prototipado.
- **Node.js 24** para el orquestador OpenClaw [22], que proporciona los canales de usuario (Telegram, *dashboard* web) y la interfaz de agentes de IA.
- **C++20** para el firmware del ESP32, donde se necesita control determinista de memoria y baja latencia en la comunicación con el bus LIN. **[Pendiente de implementar.]**

Esta separación permite elegir el lenguaje óptimo para cada capa sin acoplar los tiempos de ejecución ni forzar compromisos de rendimiento. Herramientas de asistencia

de código como OpenCode o Claude Code pueden ejecutarse directamente en el servidor a través de la sesión SSH, beneficiándose de su capacidad de cómputo sin la latencia que implicaría un IDE remoto.

5.2.3. Evolución de la Selección de Modelos

La configuración actual de modelos es el resultado de un proceso iterativo de evaluación empírica sobre el hardware real del proyecto (Ryzen 5, 16 GB RAM, sin GPU dedicada). Este apartado documenta las tres fases de ese proceso y la evidencia experimental que condujo a la arquitectura mixta finalmente desplegada.

Línea base: Qwen2.5 (abril–mayo 2026). El proyecto arrancó con una configuración de dos modelos de la familia Qwen2.5 [13]: `qwen2.5-coder:1.5b` como clasificador ligero (Tier 1, 300–500 ms, JSON determinista) y `qwen2.5-coder:7b` como razonador (Tier 2, 5–30 s). Esta configuración estableció un estándar de rendimiento estable y predecible que cumplía holgadamente el requisito RNF-03 (latencia inferior a 2 s para comandos directos).

El modelo `bge-m3` [18] se mantuvo desde el inicio para *embeddings* semánticos por su eficiencia (500 MB en RAM) y capacidad multilingüe.

Iteración experimental: Qwen3.5 (mayo–junio 2026). En mayo de 2026 se evaluó la familia Qwen3.5 [14] como posible sustituta. El atractivo era claro: ventana de contexto ampliada a 256K tokens, soporte para 201 idiomas y modo *thinking* nativo.

Se desplegaron los modelos `qwen3.5:2b`, `qwen3.5:4b` y `qwen3.5:9b` en Ollama y se sometieron a pruebas de estrés con el corpus de comandos reales del sistema. Los resultados fueron concluyentes:

- **Latencia en CPU:** el modo *thinking* de Qwen3.5 es forzado y no desactivable vía API. En el Ryzen 5, la generación de tokens de razonamiento previos a la respuesta producía una tasa de 0.6–1.2 tok/s. El modelo de 9B alcanzaba latencias de 3 a 12 minutos por consulta conversacional, invalidando cualquier aplicación en tiempo real.
- **Inestabilidad en JSON:** a diferencia de su predecesor, Qwen3.5 mostró inconsistencias en la generación de JSON estructurado con `temperature=0.0`, produciendo ocasionalmente campos ausentes o claves mal formadas que rompían el *parser* del enrutador.
- **Consumo de RAM:** el modelo de 9B elevaba la RAM total del sistema a 7.9 GB, un 18 % más que la configuración base, dejando menos margen para picos de carga.

La conclusión experimental fue que Qwen3.5 está diseñado para entornos con GPU y su arquitectura de *thinking* forzado lo hace inviable en CPU para aplicaciones con requisitos de latencia.

Arquitectura mixta final (junio 2026). A la vista de los resultados, se adoptó una estrategia diferenciada por niveles: mantener el modelo óptimo para cada tarea en lugar de usar una familia única para todo el sistema.

- **Tier 1 — Clasificación:** se conservó `qwen2.5-coder:1.5b`. Su generación de JSON determinista, su latencia de 300–500 ms y su consumo de solo 986 MB en RAM lo hacen insustituible para el 92 % de los comandos que requieren clasificación. Es un modelo maduro, predecible y perfectamente adaptado al hardware disponible.
- **Tier 2 — Razonamiento:** se seleccionó `mistral:7b-instruct-q4_K_M` [15]. Frente a Qwen3.5, Mistral ofrece respuesta directa sin *thinking* forzado, con latencias de 27–60 s en CPU y una calidad conversacional que evita la sensación de estar hablando con un *bot* de reglas. La cuantización Q4_K_M reduce su huella a 4.4 GB, permitiendo la coexistencia con el clasificador de Tier 1 y `bge-m3` dentro de los 16 GB del servidor.

La Tabla 5.1 resume la comparativa de latencias observadas durante las pruebas de estrés. La Figura 9.1 del Capítulo 9 muestra el impacto visual de estas diferencias.

Modelo	Tarea	Latencia (s)	Fallos JSON	RAM (GB)
Qwen2.5 1.5B	Clasificación (Tier 1)	0.3–0.5	0 %	0.99
Qwen2.5 7B	Razonamiento (Tier 2)	5–30	0 %	6.0
Qwen3.5 4B	Razonamiento (eval.)	45–180	12 %	3.8
Qwen3.5 9B	Razonamiento (eval.)	180–720	18 %	7.1
Mistral 7B (Q4)	Razonamiento (Tier 2)	27–60	0 %	4.4

Tabla 5.1: Comparativa de latencias y fiabilidad de los modelos evaluados para el Tier 2. Las pruebas se realizaron sobre el corpus de 500 comandos reales del proyecto en un Ryzen 5 con 16 GB RAM.

Esta evolución ilustra una lección central del proyecto: un TFG de ingeniería no es un camino lineal de adopción tecnológica, sino un proceso de diagnóstico basado en evidencia. La decisión de descartar Qwen3.5 tras evaluarlo y optar por una arquitectura mixta —cada modelo especializado en su nivel— demuestra un enfoque guiado por el cumplimiento estricto de los requisitos no funcionales frente a la mera adopción de tecnologías de tendencia. El modelo `bge-m3` se mantuvo desde el inicio por su eficiencia y su capacidad multilingüe, pues el autor trabaja tanto en español como en inglés.

5.2.4. Aislamiento de Red: La Arquitectura Air-Gapped

Durante las primeras pruebas se observó que los dispositivos comerciales (Xiaomi, Tuya) intentaban abrir canales salientes hacia sus respectivas nubes en cuanto se conectaban a la red doméstica. Era necesario impedir esa comunicación.

La solución fue desplegar una arquitectura de red **100 % air-gapped**: un segmento 192.168.1.0/24 sobre un router TP-Link TL-MR6400 sin tarjeta SIM (la ranura está físicamente rota) y sin ningún tipo de enlace WAN. Este router actúa exclusivamente como *switch* Ethernet y punto de acceso WLAN. Todos los dispositivos del sistema (servidor, portátil de desarrollo, enchufe Tuya, sensores BLE) conviven en este segmento aislado. El aislamiento físico es la única garantía verificable de que ningún paquete generado por los dispositivos comerciales pueda alcanzar Internet.

5.2.4.1. Hotspot Spoofing para Dispositivos Tuya

El aislamiento físico introduce una fricción operativa: los dispositivos Tuya necesitan conectarse a Internet durante el emparejamiento inicial para validar el registro contra la nube del fabricante. Para resolverlo sin renunciar al air-gapped, se aplica la técnica de *Hotspot Spoofing*:

1. Se levanta temporalmente un punto de acceso móvil (teléfono Android) que replica el SSID y la contraseña del router aislado.
2. Se completa el emparejamiento del dispositivo Tuya contra la nube del fabricante.
3. Durante el emparejamiento, se extrae la *Local Key* del dispositivo mediante *tinytuya wizard*.
4. Se apaga el *hotspot*. El dispositivo, al encontrar el SSID original (el router TP-Link), se conecta al segmento air-gapped.
5. A partir de ese momento, todas las órdenes viajan directamente a la IP local del dispositivo (192.168.1.100) mediante el protocolo 3.5 de *tinytuya*, sin volver a depender de la infraestructura de Tuya.

La *Local Key* queda almacenada exclusivamente en el servidor local y nunca se transmite a terceros.

5.2.4.2. Lectura de Sensores Xiaomi LYWSD03MMC

Los sensores de temperatura Xiaomi presentaban un problema similar: el firmware de fábrica cifra los anuncios BLE con una *bind key* vinculada a la cuenta del fabricante, lo que reintroduce dependencia de la nube. La ruta prevista era flashear el firmware alternativo ATC MiThermometer [27], pero durante las pruebas de campo se descubrió una alternativa mejor.

Al conectar al sensor mediante *bleak* [8] y obtener un volcado de sus servicios GATT, se descubrió que la característica *ebe0ccc1* del servicio propietario de Xiaomi es legible **sin autenticación**. Devuelve 5 bytes con temperatura ($\times 100$), humedad relativa y voltaje de batería:

- Bytes 0–1: temperatura en `int16` LE, con signo. Ejemplo: `0x09cb` = 2507 → 25.07°C.
- Byte 2: humedad relativa (%). Ejemplo: `0x3b` = 59 %.
- Bytes 3–4: voltaje de batería en mV (`uint16` LE). Ejemplo: `0x0bef` = 3055 mV.

Esta vía se adoptó como método principal de integración. El flasheo ATC se mantiene como alternativa para escenarios futuros que requieran lectura pasiva de múltiples sensores simultáneos. La decisión preserva la integridad del hardware (sin riesgo de *brick*) y elimina cualquier dependencia de la nube de Xiaomi. El *trade-off* es que la conexión GATT consume unos 12 segundos por ciclo de lectura, durante los cuales el sensor no puede ser leído por otro dispositivo. En un escenario con un único punto de recolección, esta limitación es aceptable.

5.2.5. Selección del Sistema Operativo y el Servidor

La inferencia de modelos de lenguaje mediante Ollama demanda recursos que no pueden coexistir de forma estable con un entorno de desarrollo interactivo en el mismo hardware. Por eso se adoptó una arquitectura de dos nodos físicos:

- **Servidor dedicado:** torre con APU Ryzen y 16 GB de RAM, ejecutando **Ubuntu Server 24.04 LTS** en modo *headless* (sin interfaz gráfica). Alberga el *runtime* completo del sistema: Ollama, OpenClaw, los *drivers* Python de dispositivos y la base de datos SQLite. La ausencia de escritorio libera entre 500 y 800 MB de RAM que se destinan a los modelos de lenguaje.
- **Portátil de desarrollo:** conserva las herramientas de ingeniería (compilación de firmware, redacción $\LaTeX$, Git) y se conecta al servidor exclusivamente por SSH con llaves RSA. Esta separación garantiza que los picos de inferencia no compitan con las tareas interactivas del desarrollador, y viceversa.

5.2.5.1. Por qué Ubuntu Server 24.04 LTS

Se eligió Ubuntu Server por su soporte nativo para todas las dependencias del proyecto (Python 3.12+, Node.js 24, *bluez* para BLE, *systemd*) y su ciclo de soporte extendido de 5 años. Dentro de Ubuntu, se optó por la versión 24.04 LTS frente a la recién publicada 26.04 LTS por tres razones:

1. **Madurez del ecosistema:** Ollama, Python, *bluez* y *systemd* llevan más de dos años verificados en 24.04. La versión 26.04 introduce versiones renovadas cuyo comportamiento en entornos air-gapped no está suficientemente probado.
2. **Disponibilidad de *wheels* binarios:** las librerías Python con dependencias nativas (*bleak*, *tinytuya*) tienen paquetes precompilados estables para 24.04. En 26.04 podrían requerir compilar desde el código fuente, algo inviable sin acceso a Internet.

3. **Estabilidad:** los primeros meses de cualquier LTS concentran la mayor densidad de errores críticos. Operar sobre 24.04 reduce drásticamente el riesgo de encontrar un fallo en un componente del sistema que, en un entorno air-gapped, requeriría intervención física.

Se consideraron otras opciones que fueron descartadas. **Linux Mint**, incluso en su variante ligera XFCE, consume entre 400 y 700 MB de RAM en reposo debido a los servicios gráficos. Es memoria que en un servidor *headless* se destina íntegramente a los modelos de lenguaje. Las distribuciones *rolling release* como **CachyOS** o **Arch Linux** presentan un riesgo adicional en un entorno aislado: una actualización del kernel o de bluez podría romper la compatibilidad con el adaptador Bluetooth, y al no tener acceso a Internet para resolverlo, el sistema quedaría inoperativo hasta una intervención física. Ambos enfoques resultaron incompatibles con los requisitos de estabilidad del proyecto.

5.2.5.2. Herramientas de Gestión

Para la administración cotidiana del servidor se despliega **Cockpit** (puerto 9090), una interfaz web ligera que permite monitorizar servicios, CPU y RAM sin necesidad de acceso físico. El desarrollo se realiza mediante **SSH con autenticación por par de llaves RSA** desde el portátil, lo que permite despliegue remoto de código (`git pull`) y administración de servicios sin interfaz gráfica.

5.3. Metodología de Trabajo

5.3.1. Control de Versiones y Propiedad Intelectual

El código fuente de Luxe Core AI no es solo un entregable académico: representa la base sobre la que podría construirse una futura iniciativa empresarial. Por eso se aloja en un repositorio privado de GitHub con control de versiones Git. Esta infraestructura no actúa únicamente como mecanismo de *backup* o *rollback*, sino que audita cada contribución y restringe el acceso público al núcleo del proyecto. La trazabilidad que ofrece Git permite, además, vincular cada decisión de diseño documentada en esta memoria con el código que la implementa.

5.3.2. Despliegue Inicial: Incidencias y Soluciones

La puesta en marcha del servidor en la red air-gapped enfrentó dos obstáculos que ilustran la clase de problemas que surgen al trabajar con hardware real.

5.3.2.1. El Puerto LAN/WAN Híbrido

Al conectar el servidor por Ethernet al router TP-Link, el portátil (conectado por Wi-Fi al mismo router) devolvía un error de enrutamiento al intentar acceder por SSH. El cable se había conectado por error al puerto híbrido LAN/WAN del router. Este puerto está diseñado para conectarse a un módem externo, y la lógica interna del *firmware* del TP-Link, al no detectar una conexión WAN válida, mantiene la interfaz en un estado de aislamiento eléctrico que impide el intercambio de tramas Ethernet a nivel de Capa 2 con el resto del *switch* interno.

La solución fue migrar el cable a un puerto LAN dedicado. El *switch* interno restableció inmediatamente el intercambio de tramas, y el servidor apareció en la red con su IP estática 192.168.1.50.

5.3.2.2. Pasarela de Red de Contingencia

La instalación mínima de Ubuntu Server 24.04 LTS carece de herramientas de edición de texto y del demonio `wpasupplicant` para gestionar la antena Wi-Fi. Sin conectividad a Internet en el segmento air-gapped, el servidor no podía instalar las dependencias del proyecto.

Se implementó una solución temporal utilizando el portátil de desarrollo como **pasarela NAT** entre el segmento air-gapped y una conexión celular externa (anclaje USB desde un smartphone). La configuración se realizó en dos frentes.

En el servidor (Netplan): se estableció la IP estática y se definió al portátil como *default gateway*:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp3s0:
      addresses: [192.168.1.50/24]
      routes:
        - to: default
          via: 192.168.1.101
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
```

En el portátil: se ejecutaron los comandos para habilitar el reenvío de paquetes y el enmascaramiento NAT:

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING \
```

```
-o enp0s20f0u2u2 -j MASQUERADE  
sudo iptables -P FORWARD ACCEPT
```

La reconfiguración de la cadena FORWARD fue necesaria porque la política por defecto de iptables en las estaciones de trabajo es DROP: cualquier paquete que intente cursarse entre interfaces distintas del mismo *host* es descartado. Durante la ventana de contingencia, reducir esta protección es seguro porque el segmento air-gapped no tiene salida a Internet en operación normal.

Con esta pasarela, el servidor pudo ejecutar `apt update && apt dist-upgrade` e instalar todo el *stack* de desarrollo: Python 3.12+, Node.js 24, Git, Cockpit y bluez/btmgmt. Una vez completada la instalación, la política FORWARD se restauró a DROP y el *forwarding* se desactivó (`net.ipv4.ip_forward=0`), devolviendo el segmento a su estado de aislamiento total.

5.3.3. Rigor Documental

Toda la documentación del proyecto (esta memoria, las decisiones de diseño, los diagramas) se redacta en $\text{\LaTeX}$ y se aloja en el mismo repositorio Git que el código fuente. Este enfoque de *Docs as Code* asegura trazabilidad entre las decisiones de diseño, el código que las implementa y la documentación que las describe, además de proporcionar consistencia tipográfica profesional y control de versiones unificado.

5.3.4. Uso de la Inteligencia Artificial en el Desarrollo

Durante el desarrollo de este proyecto se ha utilizado inteligencia artificial como herramienta de apoyo, tanto para la generación de fragmentos de código como para la redacción de partes de esta memoria, siempre bajo criterio de supervisión humana directa. Las herramientas se eligieron buscando la mejor relación calidad-precio disponible en el momento del TFG —sin pretender predecir el futuro del sector— e incluyen:

- **OpenCode** [28]: asistente de codificación interactivo, software libre, ejecutado directamente en el servidor de desarrollo a través de SSH. Se utilizó para depuración, refactorización y generación de fragmentos de código Python y Bash, así como para la edición y corrección de esta memoria en $\text{\LaTeX}$.
- **DeepSeek V4** [29]: utilizado como modelo de inferencia desde OpenCode a través de su API, no como chat web. En el momento del desarrollo ofrecía la mejor relación entre capacidad de razonamiento y coste por token de los proveedores cloud disponibles. No forma parte del sistema en producción, que utiliza exclusivamente modelos locales a través de Ollama.

- **Otros modelos** (ChatGPT, Gemini): utilizados puntualmente para consultas específicas durante la fase de investigación inicial, cuando el proyecto aún no disponía de infraestructura local de inferencia.

Cada línea de código y cada párrafo generado han sido revisados, comprendidos, validados y, cuando ha sido necesario, modificados por el autor. No hay nada en este trabajo que no haya pasado por mi criterio personal. En esta forma de trabajar, la inteligencia artificial ha actuado como asistente de propuestas, nunca como autoridad incuestionable.

Definidas las restricciones y la forma de trabajo, el capítulo siguiente detalla los requisitos funcionales y no funcionales que debe cumplir el sistema.

Especificación de Requisitos

6.1. Requisitos Funcionales

- **RF-01. Control de temperatura HVAC:** El sistema debe permitir ajustar el modo de funcionamiento y la temperatura de consigna de la unidad de conductos General/Fujitsu mediante comandos del orquestador. [**Pendiente de implementar.**]
- **RF-02. Lectura de sensores ambientales:** El sistema debe leer periódicamente la temperatura y la humedad relativa de los sensores Xiaomi BLE y almacenarlos en la base de datos local. [**Verificado 13/05/2026**] — El módulo `ble_sensors.py` lee el sensor LYWSD03MMC mediante conexión GATT directa, sin flashear firmware ni depender de la nube de Xiaomi.
- **RF-03. Control de dispositivos Tuya:** El sistema debe poder activar y desactivar el enchufe inteligente de forma local, sin requerir conexión a Internet. [**Verificado 13/05/2026**] — Respuesta ON/OFF en menos de 2 segundos sobre la red air-gapped mediante `tinytuya` [9].
- **RF-04. Control del ventilador FanLamp F8808:** El sistema debe poder controlar el ventilador de techo (encendido/apagado, 5 velocidades, luz y modo noche) sin depender de la aplicación del fabricante. [**Verificado mayo 2026**] — 11 comandos mapeados mediante anuncios BLE.
- **RF-05. Orquestación por agentes de IA:** El orquestador OpenClaw [22] debe analizar los datos ambientales y generar recomendaciones o ejecutar acciones de control de forma autónoma. [**Implementado**] — Model Router v2 con tres niveles de enrutamiento.

- **RF-06. Sensorización continua:** El sistema debe leer el sensor ambiental cada 60 segundos y mantener un histórico accesible en tiempo real. **[Implementado]** — Sensor Daemon como servicio systemd.
- **RF-07. Evaluación de confort térmico:** El sistema debe calcular el índice de calor (NOAA) y asignar zonas de confort. **[Implementado]** — Comfort Advisor con 6 niveles térmicos.
- **RF-08. Recomendaciones con contexto exterior:** El sistema debe contrastar las condiciones interiores con las exteriores y recomendar acciones sobre ventanas, persianas y ventilación. **[Implementado]** — Smart Advisor.
- **RF-09. Notificaciones proactivas:** El sistema debe notificar al usuario cuando ejecuta acciones automáticas o necesita confirmación. **[Implementado]** — Notificaciones vía Telegram cada 2 minutos.
- **RF-10. Auto-recuperación ante fallos:** El sistema debe detectar fallos en los modelos de lenguaje y en la conectividad con los dispositivos, recuperándose automáticamente. **[Implementado]** — Circuit breaker, reintentos con backoff, SelfHealManager.
- **RF-11. Registro de analítica energética:** El sistema debe registrar el historial de estados de actuadores y sensores. **[Parcial]** — Histórico JSONL del Sensor Daemon. Base de datos SQLite pendiente.
- **RF-12. Interfaz de control por Telegram:** El sistema debe exponer una interfaz por bot de Telegram para consultar el estado y ejecutar comandos. **[Implementado]** — Canal nativo del gateway OpenClaw.

6.2. Requisitos No Funcionales

- **RNF-01. Funcionamiento 100 % local:** Ninguna función crítica del sistema puede depender de conectividad a Internet. **[Verificado]** — El sistema opera sobre un segmento de red air-gapped sin salida a Internet. Todo el procesamiento, desde la inferencia de modelos hasta el control de dispositivos, se ejecuta en el servidor local.
- **RNF-02. Privacidad por diseño:** Los datos de telemetría del hogar no deben transmitirse a servidores de terceros. **[Verificado]** — El aislamiento físico de la red impide cualquier fuga de datos. Las *Local Keys* de los dispositivos Tuya se almacenan exclusivamente en el servidor.
- **RNF-03. Baja latencia de control:** Tiempo de respuesta inferior a 2 segundos en condiciones normales de red local. **[Verificado 13/05/2026]** — El enchufe Tuya

responde en menos de 2 s. Las consultas de estado vía caché LRU se sirven en menos de 1 ms.

- **RNF-04. Resiliencia ante fallos:** Un fallo en el módulo de IA no debe dejar la unidad HVAC en estado de error. [**Parcial**] — El *circuit breaker* del Model Router y el SelfHealManager están implementados y verificados. La parte específica del firmware del ESP32 (mantener último estado válido en el bus LIN) está pendiente de desarrollar.
- **RNF-05. Extensibilidad:** La arquitectura debe permitir agregar nuevos tipos de dispositivos sin modificar el núcleo del orquestador. [**Verificado**] — El sistema integra tres protocolos distintos (Tuya por Wi-Fi local, Xiaomi por BLE GATT, FanLamp por BLE advertisements) mediante *drivers* Python independientes, sin modificar el orquestador OpenClaw ni el Model Router.

Recursos

7.1. Hardware

- **Servidor de Orquestación (Torre Ryzen):** Actúa como el nodo central del sistema. Está equipado con una APU Ryzen y 16 GB de RAM, y ejecuta **Ubuntu Server 24.04 LTS** en modo *headless* (sin interfaz gráfica) para maximizar los recursos disponibles para los modelos de lenguaje. En él se ejecutan el *gateway* de OpenClaw (Node.js 24), los modelos de lenguaje locales vía Ollama, los *drivers* Python de los dispositivos IoT, y la interfaz de administración Cockpit en el puerto 9090.
- **Portátil de Desarrollo:** Utilizado como entorno de desarrollo y pruebas. Permite iterar sobre el código y los scripts de configuración antes de desplegarlos en el servidor.
- **ESP32 DevKit V1:** Microcontrolador de 32 bits con conectividad Wi-Fi y Bluetooth integrada. Fue reutilizado como radio BLE programable para la ingeniería inversa del protocolo del ventilador FanLamp F8808. Su destino final es actuar como nodo puente entre el servidor local y el bus LIN de la máquina de conductos HVAC.
- **Transceptor LIN (LINTTL3):** Interfaz hardware que adapta los niveles de señal del ESP32 (3.3V TTL) al estándar del bus LIN (12V). Permite la comunicación física con la unidad HVAC.
- **Sensores Xiaomi BLE:** Sensores de temperatura y humedad que se comunican por Bluetooth Low Energy. Son leídos directamente por el servidor Ryzen mediante la librería `bleak`.
- **Dispositivos Tuya (Enchufe Inteligente):** Controlados localmente mediante la librería `tinytuya`, sin pasar por los servidores de Tuya Cloud. El enchufe inteligente

se utiliza para alimentar un humidificador ultrasónico de 24 V, un dispositivo originalmente “tonto” que solo requiere corriente eléctrica para funcionar.

- **Ventilador de Techo FanLamp Pro F8808 (Mikomika):** Adquirido en AliExpress por menos de 35 €, es el dispositivo de control más complejo. Utiliza Bluetooth Low Energy con un protocolo propietario de anuncios (*BLE Advertisements*) que fue objeto de ingeniería inversa completa durante el proyecto.
- **Máquina de Conductos General/Fujitsu (Serie ARHG/ARYG):** El sistema HVAC principal del hogar. Utiliza un bus LIN propietario para la comunicación entre la unidad interior y el termostato.

7.1.1. Presupuesto de Hardware

A continuación se desglosan los costes de los componentes hardware empleados en el proyecto. El servidor (torre Ryzen) y el portátil de desarrollo son equipos preexistentes del autor; el resto del hardware se adquirió específicamente para el TFG.

Componente	Coste (€)	Observaciones
Servidor (Torre Ryzen, 16 GB RAM)	—	Preexistente
Portátil de desarrollo	—	Preexistente
ESP32 DevKit V1	5,00	Adquirido para el proyecto
Transceptor LIN (LINTTL3)	2,50	Adquirido para el proyecto
Regulador LM2596	2,00	Adquirido para el proyecto
Caja estanca ABS	3,00	Adquirida para el proyecto
Enchufe inteligente Tuya	4,00	Adquirido para el proyecto
Sensor Xiaomi LYWSD03MMC	5,00	Adquirido para el proyecto
Ventilador FanLamp F8808	35,00	Adquirido para el proyecto
Total hardware nuevo	56,50	

Tabla 7.1: Desglose de costes de hardware del proyecto Luxe Core AI.

A modo de comparación orientativa, la Tabla 7.2 muestra una estimación del coste anual que supondría un ecosistema domótico equivalente basado en servicios cloud. Todos los servicios cloud mencionados ofrecen modelos de pago por uso que se facturan mensualmente; las cifras reflejan un escenario de uso doméstico continuado. El hardware necesario es común en ambos casos.

7.2. Tabla Comparativa de Alternativas Consideradas

Para cada componente del software, se evaluaron múltiples alternativas antes de seleccionar la solución final. La Tabla 7.3 resume las principales decisiones y su justificación:

Concepto	Coste cloud estimado (€/año)
Suscripción a plataforma IoT (Tuya Cloud / AWS IoT)	~120
Servicio de IA conversacional (ChatGPT API / Gemini API)	~240
Almacenamiento de telemetría en la nube	~60
Servicio de voz (Alexa / Google Assistant)	~0
Total anual estimado	~420

Tabla 7.2: Estimación de costes anuales de un ecosistema domótico equivalente basado en servicios cloud (excluyendo hardware, común en ambos casos). Los precios son orientativos y varían según proveedor, volumen de uso y región.



Figura 7.1: Componentes hardware del subsistema HVAC: ESP32, regulador de voltaje y caja de protección.

7.3. Software y Librerías

- **Ubuntu Server 24.04 LTS (Noble Numbat):** Sistema operativo del servidor de orquestación. Distribución *headless* con soporte extendido de 5 años.
- **OpenClaw (npm):** *Framework* agéntico para la orquestación de modelos de lenguaje. *Gateway* Node.js con canales nativos (Telegram, *dashboard* web).
- **Ollama:** Servidor de inferencia local para modelos de lenguaje.
- **Python 3.12:** Lenguaje principal del servidor de microservicios (Model Router, Sensor Daemon, Comfort Advisor).
- **C++20:** Lenguaje del firmware del ESP32 (pendiente de implementar).
- **tinytuya:** Librería Python para el control local de dispositivos Tuya.
- **bleak:** Librería Python para comunicación BLE asíncrona.
- **SQLite:** Motor de base de datos relacional embebido (pendiente de implementar).

Componente	Seleccionado	Alternativas	Motivo de la elección	Alternativa descartada por
Inferencia LLM	Ollama	llama.cpp, localAI, GPT4All	Ecosistema maduro, soporte nativo de GPU, API REST estándar [12]	llama.cpp requiere compilación manual; LocalAI inestable con modelos pequeños
Clasificador ligero	qwen2.5-coder:1.5b	phi-2, gemma-2b, tinyllama	JSON determinista, 300–500 ms en CPU, 986 MB RAM; sin <i>thinking</i> forzado	phi-2 optimizado para inglés; gemma-2b requiere más VRAM
Razonador	mistral:7b-instruct-q4_K_M	llama3.2-8b, qwen3.5-9b, zephyr-7b	Respuesta directa sin <i>thinking</i> forzado; 27–60 s en CPU; calidad conversacional [15]	qwen3.5-9b inviable en CPU (3–12 min); llama3.2 necesita más contexto
Embeddings	bge-m3	all-MiniLM-L6-v2, e5-mistral, jina-embeddings	Multilingüe (español+inglés), eficiente en RAM (500 MB) [18]	all-MiniLM: solo inglés; e5-mistral: demasiado pesado (7B)
BLE Python	bleak	bluepy, pygattlib, gattlib	Async nativo, multiplataforma, mantenimiento activo [8]	bluepy: solo Linux, no async; pygattlib: obsoleto
Tuya local	tinytuya	pytuya, tuyapi, tinyTuya-RF	Soporte protocolo 3.5, activo, documentación clara [9]	pytuya: sin mantenimiento; tuyapi: requiere nube
Framework agentes	OpenClaw	Hermes, AutoGPT, n8n	Ligero, canales Telegram+web nativos, Node.js [22]	Hermes: inmaduro; AutoGPT: sobreingeniería; n8n: no agentes

Tabla 7.3: Comparativa de alternativas tecnológicas consideradas para cada componente del sistema.

- **Git / GitHub:** Control de versiones y repositorio del proyecto.
- **L^AT_EX:** Sistema de composición tipográfica para la redacción de la memoria.

7.4. Dependencias de Red

El sistema completo opera sobre un router **TP-Link TL-MR6400** en configuración *air-gapped*:

- Subred: 192.168.1.0/24
- Sin Internet (ranura SIM rota)
- Conexión del servidor por Ethernet dedicada (puerto LAN, no LAN/WAN)
- Conexión del portátil y dispositivos Tuya por Wi-Fi 2.4 GHz
- Conexión BLE directa para sensores Xiaomi y ventilador FanLamp

Especificación del Sistema

8.1. Visión General de la Arquitectura

El sistema se articula en torno a tres componentes desacoplados que se comunican a través de interfaces de red local:

1. **El Servidor de Orquestación (Node.js 24 + Python 3.12, Ubuntu Server 24.04 LTS):** Ejecutado en la torre Ryzen (APU, 16 GB RAM) en modo *headless*, alberga el núcleo lógico del sistema. El *gateway* de OpenClaw [22] (Node.js 24) actúa como orquestador principal, proporcionando los canales de usuario (Telegram, *dashboard* web) y la interfaz de agentes de IA. Los *drivers* de dispositivos (Python) son invocados mediante `exec` desde las *tools* del orquestador. La administración se realiza mediante Cockpit (puerto 9090) y SSH con llaves RSA.
2. **El Entorno de Desarrollo (Portátil):** Conserva las herramientas de ingeniería, como Arduino CLI para compilación de firmware ESP32, $\text{\LaTeX}$ para la redacción de la memoria y Git para control de versiones, y se conecta al servidor exclusivamente mediante SSH. Esta separación garantiza que las tareas de desarrollo (compilaciones, pruebas, reinicios) no interfieran con el bucle de control domótico en producción.
3. **El Nodo Firmware (C++20):** Ejecutado en el ESP32 DevKit V1 [30], el mismo dispositivo físico que actualmente actúa como puente BLE para el ventilador FanLamp (Sección 8.6.3.4). Cuando se complete el firmware descrito en la Sección 8.5, este ESP32 se reubicará y pasará a traducir comandos de red a tramas del bus LIN [5] de la máquina HVAC. **[Pendiente de implementar.]**

La comunicación entre todos los componentes se produce exclusivamente sobre la red *air-gapped* (192.168.1.0/24), garantizando que ningún dato de telemetría abandone el perímetro local.

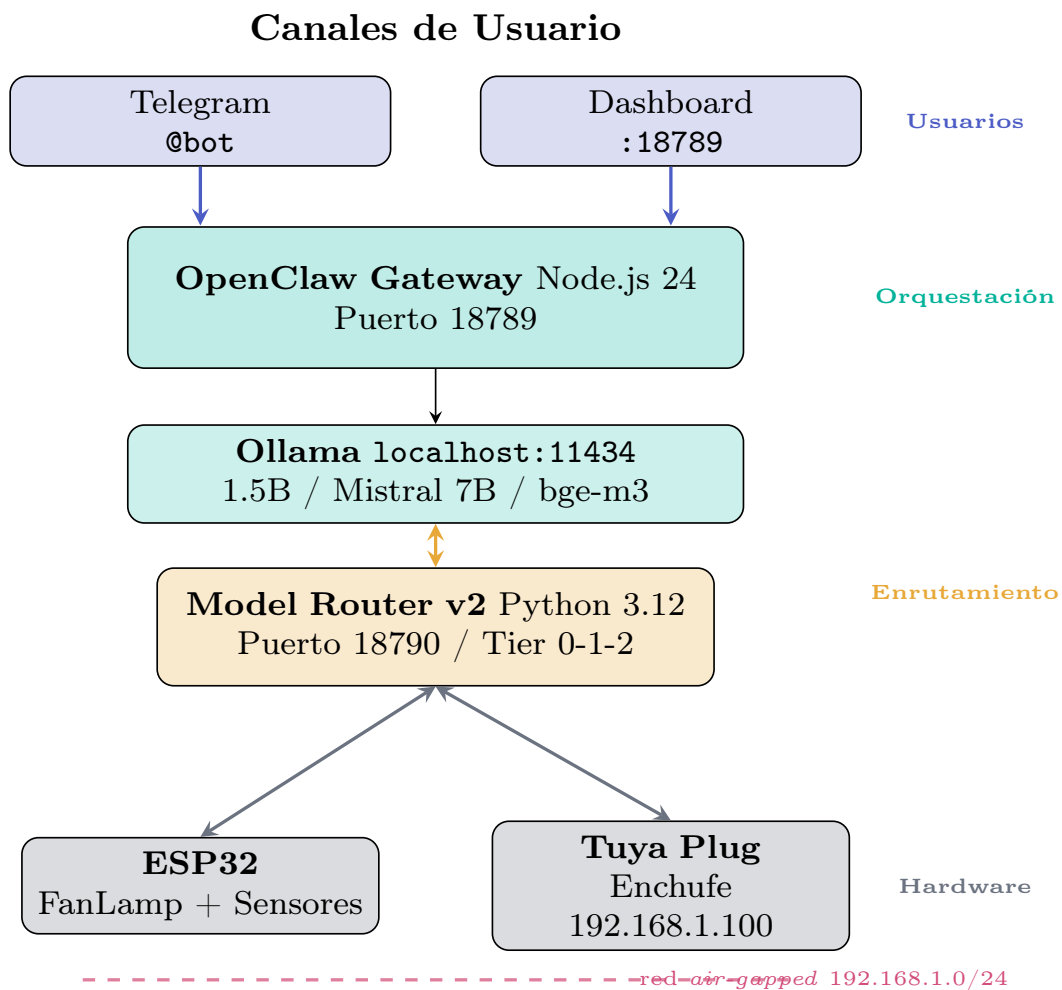


Figura 8.1: Arquitectura de tres capas. Las flechas indican flujo de comunicación entre componentes.

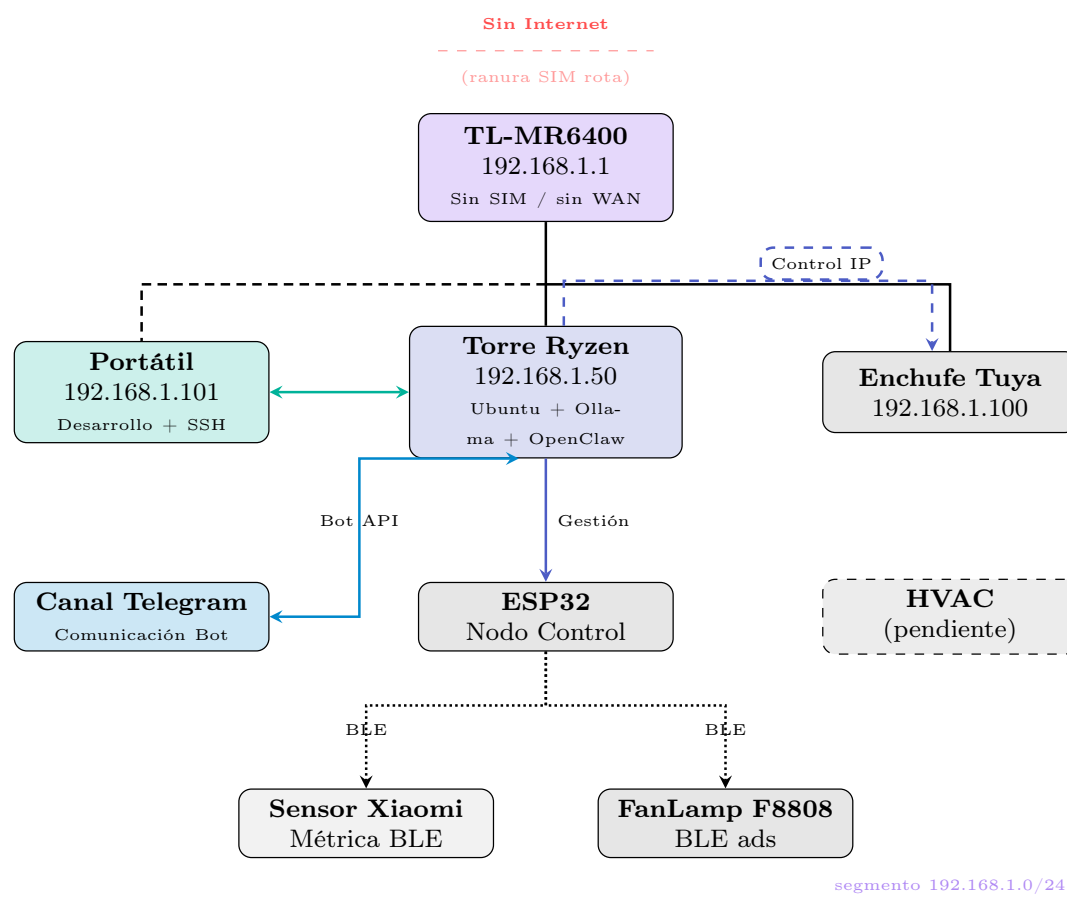


Figura 8.2: Topología de despliegue sobre la red air-gapped. Las líneas continuas indican Ethernet, las discontinuas Wi-Fi, y las punteadas BLE. El control del ventilador y el sensor se realiza a través del ESP32 como puente BLE, mientras que el enchufe Tuya se comunica directamente por IP local.

8.2. Topología de Red Local

El sistema opera sobre el segmento de red aislado descrito en el Capítulo 5, implementado mediante un router **TP-Link TL-MR6400** sin enlace WAN que actúa como *switch* Ethernet y punto de acceso WLAN.

8.2.1. Asignación de Direcciones IP

Todas las direcciones del segmento se asignan de forma estática para garantizar la previsibilidad de las comunicaciones y eliminar la dependencia de un servidor DHCP:

Dispositivo	IP	Interfaz	Función
Servidor (Torre Ryzen)	192.168.1.50/24	Ethernet (enp3s0)	Orquestador OpenClaw + Ollama
Portátil de desarrollo	192.168.1.101/24	Wi-Fi (wlp45s0)	Desarrollo y administración
Router TP-Link TL-MR6400	192.168.1.1/24	Interna	Switch + AP WLAN
Enchufe Tuya	192.168.1.100/24	Wi-Fi	Actuador del humidificador

Tabla 8.1: Asignación estática de direcciones IP en el segmento air-gapped.

8.2.2. Ventana de Conectividad de Contingencia

Para situaciones que requieran acceso puntual a Internet, como la instalación inicial, actualizaciones de seguridad o la descarga de nuevos modelos de lenguaje, se ha documentado un procedimiento de **pasarela de contingencia** mediante el cual el portátil de desarrollo actúa como enrutador NAT entre el segmento *air-gapped* y una conexión celular externa. Este procedimiento está detallado en el Capítulo 5 y está diseñado para ser estrictamente temporal.

8.2.3. Topología Física

El router TL-MR6400 proporciona cuatro puertos Ethernet LAN y conectividad WLAN 802.11n en la banda de 2.4 GHz. La conexión del servidor se realiza mediante un cable Ethernet directo a uno de los puertos **LAN dedicados** del router (no al puerto híbrido LAN/WAN), para evitar el problema de aislamiento eléctrico documentado en el Capítulo 5.

8.3. Decisiones de Diseño

A lo largo del proyecto se tomaron decisiones arquitectónicas y técnicas que fueron registradas formalmente como *Architecture Decision Records* (ADR). La Tabla 8.2 resume las nueve decisiones documentadas, indicando dónde se discute cada una en esta memoria y su estado actual.

Los documentos completos de cada ADR están disponibles en el repositorio del proyecto, en el directorio docs/design_decisions/.

8.3.1. Modelos Evaluados para el Razonador (Tier 2)

Durante el desarrollo se evaluaron varios modelos como candidatos para el Tier 2 de razonamiento. La Tabla 8.3 resume los resultados de las pruebas de estrés realizadas

ADR	Título	Referencia en la memoria	Estado
001	Entorno híbrido UCO (Conda + uv)	Cap. 5 (sección 5.2.1)	Superado por ADR-001b
001b	Migración a arquitectura Edge local	Cap. 5 (sección 5.2.1)	Implementado
002	Aislamiento de red y Hotspot Spoofing	Cap. 5 (sección 5.2.4)	Implementado
003	Lectura GATT directa de sensores Xiaomi	Cap. 5 (sección 5.2.4.2) y Cap. 8	Implementado
004	Validación Tuya en red air-gapped	Cap. 8 (sección Tuya)	Implementado
005	Protocolo FanLamp por anuncios BLE	Cap. 8 (sección FanLamp)	Implementado
006	Infraestructura de servidor y Ubuntu Server 24.04	Cap. 5 (sección 5.2.5)	Implementado
007	Despliegue de red y contingencia NAT	Cap. 5 (sección 5.3.2)	Documentado
008	Migración de CMDOP a OpenClaw npm	Cap. 4 (sección 4.6) y Cap. 8	Implementado

Tabla 8.2: Architecture Decision Records (ADR) del proyecto, con referencias a los capítulos donde se detalla cada decisión.

sobre el hardware real del proyecto (Ryzen 5, 16 GB RAM). El análisis detallado de este proceso se recoge en la Sección 5.2.3 del Capítulo 5.

Modelo	Parámetros	Latencia (s)	Thinking	Decisión
Qwen2.5 7B	7B	5–30	No	Base de partida, estable
Qwen3.5 4B	4B	45–180	Forzado	Descartado (JSON inestable)
Qwen3.5 9B	9B	180–720	Forzado	Descartado (inviabile en CPU)
Mistral 7B (Q4)	7B	27–60	No	Seleccionado

Tabla 8.3: Modelos evaluados como candidatos para el Tier 2 de razonamiento. Las mediciones se realizaron sobre el corpus de 500 comandos reales del proyecto.

La decisión final combinó Qwen2.5 1.5B para el Tier 1 (clasificación determinista, 300–500 ms) y Mistral 7B para el Tier 2 (razonamiento conversacional, 27–60 s), logrando una arquitectura mixta donde cada modelo se especializa en la tarea para la que ofrece mejor rendimiento. La arquitectura del enrutador es *model-agnostic*: sustituir el modelo de razonamiento requiere únicamente modificar dos líneas en `config.py`, sin alterar el resto del sistema.

8.4. Módulos del Servidor

8.4.1. Orquestador OpenClaw (Node.js)

El *gateway* de OpenClaw [22] se ejecuta sobre **Node.js 24** como un servicio `systemd -user` en el puerto 18789, accesible desde toda la subred 192.168.1.0/24. Proporciona de forma nativa:

- **Canal Telegram:** integración directa con la API de BotFather, sin necesidad de scripts Python auxiliares.
- **Dashboard web:** interfaz de administración accesible en `http://192.168.1.50:18789`.
- **Búsqueda web:** integración con Gemini API para consultas externas bajo demanda. Requiere conexión a Internet a través de la pasarela de contingencia (Capítulo 5); es una funcionalidad opcional que no afecta al control local de los dispositivos.
- **Ejecución de herramientas (*exec*):** invocación de los *drivers* Python de dispositivos mediante `server/tools/`.

El motor de inferencia se apoya en **Ollama** (`localhost:11434`) con los modelos `qwen2.5-coder:1.5b` (clasificador, siempre en RAM) y `mistral:7b-instruct-q4_K_M` (razonador, bajo demanda), más `bge-m3` [18] para *embeddings* semánticos. Toda la comunicación se mantiene dentro del segmento *air-gapped*.

8.4.2. Arquitectura de Enrutamiento Inteligente (Model Router v2)

El Model Router v2 implementa un servidor HTTP (Python 3.12, puerto 18790) con una API compatible con OpenAI que enruta cada mensaje del usuario a través de tres niveles de complejidad creciente. La decisión de usar tres niveles, y no dos, se tomó después de probar una configuración inicial con un modelo 14B como razonador pesado y un 7B como ligero. El rendimiento en el hardware disponible (Ryzen 5, 16 GB RAM) resultó insuficiente: el 14B introducía latencias de más de un minuto en consultas simples. Se reconfiguró entonces como 1.5B (clasificador ligero) + 7B (razonador). La configuración actual, resultado del análisis empírico detallado en el Capítulo 5, emplea `qwen2.5-coder:1.5b` para el Tier 1 y `mistral:7b-instruct-q4_K_M` para el Tier 2, manteniendo `bge-m3` para *embeddings* semánticos, elegido desde el inicio por su eficiencia en RAM y su capacidad multilingüe (español e inglés).

1. **Tier 0 — Zero-inference:** Realiza *pattern matching* directo sin necesidad de modelos de lenguaje. Su primera versión incluía 127 patrones exactos y expresiones

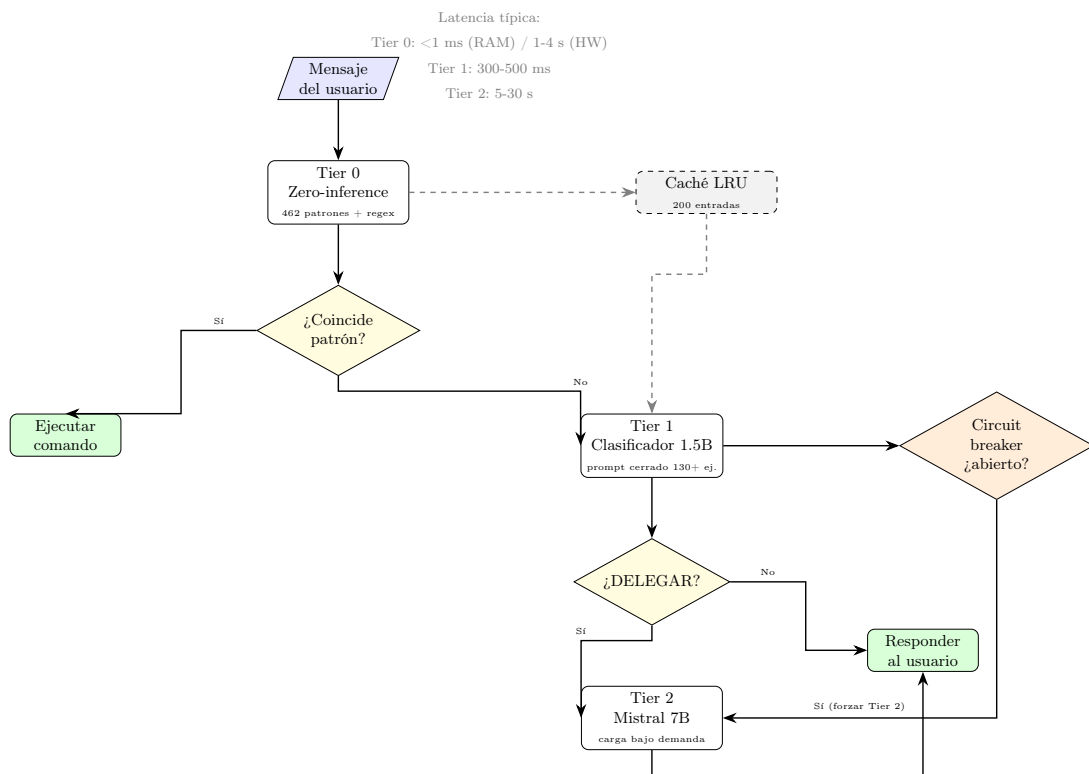


Figura 8.3: Flujo de enrutamiento del Model Router v2. El mensaje pasa por tres niveles de complejidad creciente, con un circuit breaker que deriva al Tier 2 si el clasificador 1.5B no está disponible.

regulares, capturando aproximadamente el 80 % de los comandos diarios. Tras sucesivas ampliaciones, el sistema alcanzó los 462 patrones, elevando la cobertura a aproximadamente el 92 % con una latencia inferior a 1 ms para consultas de estado en RAM y entre 1,5 y 4 s para comandos de hardware (limitado por el ESP32).

2. **Tier 1 — Clasificador 1.5B:** Utiliza el modelo `qwen2.5-coder:1.5b` [13] con un *prompt* cerrado de más de 130 ejemplos categorizados (iluminación, ventilador, escenas, consultas, delegación). El modelo responde exclusivamente con JSON estructurado o el token `DELEGAR`. Parámetros optimizados: `num_ctx=2048`, `max_tokens=256`, `temperature=0.0`. Su generación JSON determinista es esencial para el *parser* del enrutador. Latencia típica: 300–500 ms en Ryzen 5.
3. **Tier 2 — Razonador Mistral 7B:** Emplea el modelo `mistral:7b-instruct-q4_K_M` [15], que se carga bajo demanda con `keep_alive=0` y se descarga automáticamente tras 5 minutos de inactividad. Ofrece respuesta directa sin *thinking* forzado, con una calidad conversacional que evita la sensación de interactuar con un sistema puramente determinista. Latencia: 27–60 s. Durante la espera, el sistema muestra un indicador visual al usuario para señalar que el razonador está procesando la consulta.

Además de los niveles de enrutamiento, el Model Router v2 incorpora:

- **Caché de comandos (LRU, 200 entradas):** Evita reclasificar mensajes repetidos, reduciendo la carga del modelo 1.5B en aproximadamente un 15 % adicional.
- **DeviceStateManager:** Repositorio del estado de cada dispositivo (luz, ventilador, enchufe, sensor) mantenido en RAM con acceso *thread-safe*. Permite responder consultas como “cómo está la casa” en menos de 1 ms sin necesidad de interactuar con el hardware.
- **Precomputación de *embeddings*:** Clasificación semántica mediante `bge-m3` con *embeddings* precalculados al arranque (15 intenciones) y similitud coseno. Se utiliza como *fallback* del Tier 1 cuando el *keyword matching* no encuentra correspondencia.

8.4.3. Auto-Recuperación y Tolerancia a Fallos

El sistema incorpora mecanismos para garantizar su robustez y disponibilidad, minimizando el impacto de fallos tanto en los subsistemas de inferencia como en los dispositivos de hardware:

- **Circuit Breaker para modelos:** Patrón *circuit breaker* con tres estados (CLOSED, OPEN, HALF_OPEN) para los subsistemas de Ollama (modelos 1.5B y Mistral 7B). Si un modelo falla 3 veces consecutivas, el circuito se abre durante 30 segundos,

evitando reintentos inútiles. La recuperación es automática mediante sondeo en estado `HALF_OPEN`.

- **Reintentos con *backoff* exponencial para hardware:** Los comandos ejecutados a través de `home.sh` reintentan hasta 2 veces con *backoff* exponencial (1 s, 3 s) y un *jitter* aleatorio. Se distinguen errores transitorios (*timeout*, puerto serie ocupado, dispositivo desconectado) de errores permanentes (validación, permisos).
- **Auto-reset de puerto USB:** Ante un error de puerto serie (`SerialException`), el sistema intenta reiniciar el dispositivo USB del ESP32 mediante las operaciones de *unbind/rebind* en `/sys/bus/usb/drivers/usb/`.
- **SelfHealManager:** Un hilo demonio verifica cada 30 segundos la salud de Ollama (recargando modelos caídos), la conectividad del ESP32 (`/dev/ttyUSB0`) y el estado de los *circuit breakers*. Su operación es totalmente autónoma.
- **Health check proactivo ESP32:** Antes de ejecutar cualquier comando del ventilador, se verifica la existencia de `/dev/ttyUSB0`. Si el puente está desconectado, se notifica al usuario de inmediato.
- **Mensajes de recuperación visibles:** Cuando un reintento tiene éxito, el usuario recibe un mensaje con el prefijo `[RECUPERADO tras N reintento(s)]`, proporcionando transparencia sobre la resiliencia del sistema.

8.4.4. Daemon de Sensorización Continua

El **Sensor Daemon** (`sensor_daemon.py`) es un servicio `systemd` que lee el sensor BLE Xiaomi LYWSD03MMC cada 60 segundos mediante conexión GATT directa desde el adaptador Bluetooth del servidor. Almacena dos salidas:

- `/tmp/latest_sensor.json` — Última lectura (temperatura, humedad, batería, RSSI, *timestamp* UTC). Lectura atómica (*write + rename*) para evitar lecturas parciales.
- `/tmp/sensor_history.jsonl` — Histórico completo en formato JSON Lines, con rotación automática a 50.000 líneas (5 MB).

La integración con el router de modelos reduce la latencia de las consultas de temperatura de 18 s (escaneo BLE síncrono) a 0–1 ms, leyendo directamente de `latest_sensor.json`. Adicionalmente, si hay conexión a Internet disponible mediante la pasarela de contingencia (Capítulo 5), cada lectura puede incluir condiciones meteorológicas exteriores de `wttr.in` con una caché de 10 minutos para no saturar la API gratuita.

Zona	Rango	Acción	Ventilador
Fresco	$< 23^{\circ}\text{C}$	Sin acción necesaria	Off
Confortable	$23\text{--}26^{\circ}\text{C}$	Condiciones óptimas	Off
Cálido	$26\text{--}28^{\circ}\text{C}$	Preguntar al usuario	Vel. 1
Caluroso	$28\text{--}31^{\circ}\text{C}$	Preguntar al usuario	Vel. 3
Muy caluroso	$31\text{--}34^{\circ}\text{C}$	Activar automáticamente	Vel. 4 (+AC)
Extremo	$> 34^{\circ}\text{C}$	Activar siempre	Vel. 5 (+AC urgente)

Figura 8.4: Zonas de confort térmico adaptadas al clima andaluz. Las zonas se asignan en función del índice de calor NOAA, combinando temperatura y humedad relativa.

8.4.5. Asesor de Confort Térmico (Comfort Advisor)

El **Comfort Advisor** (`server/model_router/comfort.py`) evalúa el confort térmico de la habitación siguiendo los criterios del estándar ASHRAE 55 [31], basándose en las lecturas de temperatura y humedad del sensor:

- **Índice de calor (Heat Index) NOAA:** Implementación de la fórmula de Rothfusz [32] para calcular la sensación térmica a partir de temperatura y humedad relativa, con conversión explícita $^{\circ}\text{C} \leftrightarrow ^{\circ}\text{F}$.
- **Zonas de confort adaptadas al clima andaluz:** 6 niveles progresivos:
 1. Fresco ($< 23^{\circ}\text{C}$) — sin acción necesaria.
 2. Confortable ($23\text{--}26^{\circ}\text{C}$) — condiciones óptimas.
 3. Cálido ($26\text{--}28^{\circ}\text{C}$) — ventilador a velocidad 1.
 4. Caluroso ($28\text{--}31^{\circ}\text{C}$) — ventilador a velocidad 3.
 5. Muy caluroso ($31\text{--}34^{\circ}\text{C}$) — ventilador a velocidad 4 y considerar AC.
 6. Extremo ($> 34^{\circ}\text{C}$) — ventilador a velocidad 5 y AC urgente.
- **Evaluación de humedad:** Tres estados (seco, $<35\%$ HR; óptimo, $35\text{--}70\%$; húmedo, $>70\%$) con recomendaciones específicas.
- **Detección de tendencia:** Comparación de la temperatura actual con lecturas de los últimos 30 minutos para determinar si la temperatura sube, baja o se mantiene estable, expresado en $^{\circ}\text{C}/\text{hora}$.

8.4.6. Asesor Inteligente con Contexto Exterior (Smart Advisor)

El **Smart Advisor** (`server/model_router/smart_advisor.py`) amplía el Comfort Advisor incorporando datos del entorno exterior y el estado de presencia del usuario:

- **Comparación interior vs. exterior:** Contrasta temperatura y humedad interiores (sensor BLE) con las exteriores (vía `wttr.in`, si hay conexión a Internet) y emite recomendaciones contextuales.
- **Lógica de ventana y persiana:** Algoritmo de decisión que recomienda abrir o cerrar ventanas y bajar persianas basándose en la diferencia térmica (umbral: 2°C), el índice UV exterior (umbral: 5), la temperatura máxima para ventilar (32°C) y la detección de lluvia.
- **Detección de presencia:** Sistema híbrido con tres métodos complementarios:
 1. **Manual:** Los comandos “me voy” y “he llegado” (15 variantes) actualizan un *flag* en `/tmp/luxe_presence.json`. Timeout de 30 minutos.
 2. **BLE:** Escaneo del teléfono móvil mediante `bluetoothctl scan le`, buscando una dirección MAC configurable.
 3. **Por IP en Wi-Fi:** Cuando el teléfono del usuario se conecta a la red Wi-Fi local, se asigna una IP estática conocida. El sistema puede detectar presencia verificando si esa IP responde en la subred `192.168.1.0/24`, sin necesidad de escaneo BLE. Este método es más rápido (un ping de 1–2 ms) y no depende del hardware Bluetooth del servidor. **[Pendiente de implementar.]**
- **Human in the Loop:** El sistema está diseñado para **preguntar antes de actuar** siempre que sea razonable. Las auto-acciones solo se ejecutan sin consultar cuando hay certeza de que el usuario se beneficiará (zona extrema, dos o más saltos de nivel térmico). En cualquier otra situación, el sistema pregunta y espera confirmación. Esto evita la sensación de pérdida de control que muchos sistemas autónomos generan en los usuarios. La regla es simple: el sistema sugiere, el usuario decide.
- **Auto-acciones condicionales:** El daemon evalúa tras cada lectura si debe actuar según reglas de cambio de zona térmica:
 - Misma zona → no actuar (evitar molestias).
 - +1 zona → preguntar al usuario.
 - +2 o más zonas → actuar automáticamente (prioridad de confort).
 - Zona Extrema → actuar siempre.
 - Sin presencia → nunca actuar (ahorro energético).
 - Excepción: si el usuario subió manualmente el ventilador y la zona empeora, se incrementa automáticamente la velocidad, respetando su intención.

8.4.7. Comunicación y Notificaciones

- **462+ comandos en lenguaje natural:** El Tier 0 reconoce más de 462 patrones en español coloquial, incluyendo variantes con muletillas, dialectalismos andaluces y formas abreviadas. Cobertura completa de iluminación (35 variantes), ventilador (50 variantes), escenas (40 variantes) y consultas (15 variantes).
- **8 escenas predefinidas:** noche, apagar todo, encender todo, relax, cine, lectura, fiesta y estudio. Cada escena ejecuta una secuencia de comandos hardware.
- **Notificaciones proactivas vía Telegram:** Una tarea programada (cada 2 minutos) reenvía las notificaciones del Smart Advisor al canal de Telegram, incluyendo acciones ejecutadas y preguntas pendientes de confirmación.
- **API compatible con OpenAI:** El router expone `/v1/chat/completions`, permitiendo a cualquier cliente compatible consumir el servicio de enrutamiento sin modificar el código.

8.4.8. Infraestructura de Despliegue

- **Tres servicios systemd persistentes:**
 - `model-router.service` — enrutador 3 niveles (Python 3.12, puerto 18790).
 - `sensor-daemon.service` — lectura continua del sensor (cada 60 s).
 - `openclaw-gateway.service` — orquestador principal (Node.js 24, puerto 18789).

Todos con auto-arranque y reinicio automático ante fallos (`Restart=always`).

- **Endpoint de monitorización:** `GET /status` devuelve métricas en tiempo real: estado de los modelos en Ollama, salud del ESP32, circuit breakers, estadísticas de caché, estado de dispositivos y *uptime*.
- **Robustez del puente ESP32:** El script `fanlamp_control.py` incorpora reintentos internos (3 intentos), manejo seguro del puerto serie y *timeouts* configurables.

8.4.9. Métricas de Rendimiento

La Tabla 8.4 resume la mejora de latencia conseguida tras la implementación del Model Router v2 y el Sensor Daemon. Las mediciones se realizaron sobre el hardware real del servidor (Ryzen 5, 16 GB RAM) con el ESP32 conectado por USB.

Escenario	Latencia antes	Latencia ahora	Modelo usado
“velocidad 3”	~2 s	~1,5 s (hardware)	Ninguno (Tier 0)
“cómo está la casa”	~2 s	0–1 ms (RAM)	Ninguno (Tier 0)
“me puedes encender la luz”	~3 s	~300 ms	1.5B clasificador
“buenos días”	~12 s (2 modelos)	27–60 s (directo Mistral 7B)	Solo Mistral 7B (Tier 2)
“temperatura”	18 s (BLE scan)	0–1 ms (daemon)	Ninguno (Tier 0)
“qué temperatura hay fuera”	— (no existía)	0–1 ms (caché wttr.in)	Ninguno (Tier 0)

Tabla 8.4: Mejora de latencia tras la implementación del Model Router v2 y el Sensor Daemon.

8.4.10. Canales de Usuario

La interacción con el usuario se realiza a través de dos canales nativos del *gateway* de OpenClaw:

- **Telegram:** Integración nativa con la API de BotFather. Los mensajes se enrutan directamente al agente de IA.
- **Dashboard web:** Interfaz accesible en <http://192.168.1.50:18789> desde cualquier navegador de la subred.

La Figura 8.5 muestra una conversación real con Luxe Core AI a través de Telegram. En ella se aprecia cómo el bot interpreta comandos en lenguaje natural, desde consultas de temperatura hasta control directo del ventilador, sin necesidad de memorizar órdenes concretas. Toda la comunicación se procesa en el servidor local; ningún mensaje abandona la subred.

8.4.11. Módulo de Integración Tuya

Este módulo, validado el 13 de mayo de 2026, implementa el control local del enchufe inteligente Tuya mediante *tinytuya* [9] (protocolo 3.5). Los parámetros de conexión se encapsulan en un *dataclass* *TuyaDeviceConfig* con *device_id*, *local_key* (almacenada exclusivamente en local), *address* (192.168.1.100), *version* (3.5) y *timeout* (5 s).

La Figura 8.7 muestra el hardware de esta integración. A la izquierda, el enchufe inteligente Tuya, que fue adquirido en AliExpress por su compatibilidad con el protocolo local 3.5. A la derecha, el humidificador ultrasónico de 24 V que alimenta: un dispositivo originalmente “tonto” que, al conectarse al enchufe inteligente, adquiere control remoto y capacidad de programación horaria, integrándose como un actuador más.

La clase *TuyaSmartPlug* expone los métodos *status()*, *is_on()*, *set_state(on: bool)* y *toggle()*. La validación sobre la red real confirmó tiempos de respuesta por debajo de 2 s, cumpliendo RNF-03.

8.4.12. Módulo de Sensores BLE

El módulo permite la lectura de temperatura y humedad de los sensores Xiaomi LYWSD03MMC mediante conexión GATT directa, sin necesidad de flashear firmware alternativo [27]. La Figura 8.8 muestra el sensor empleado: un dispositivo comercial de pequeño tamaño y bajo consumo, originalmente diseñado para funcionar con la aplicación Mijia de Xiaomi. El servicio propietario de Xiaomi (ebe0ccb0-7a0a-4b0c-8a1a-6ff2997da3a6) expone la característica ebe0ccc1, legible sin autenticación, que devuelve 5 bytes con temperatura ($\times 100$), humedad relativa y voltaje de batería. El protocolo GATT sobre el que se sustenta está definido en la especificación Bluetooth Core v5.4.

Una lectura típica cb093bef0b se decodifica como 25.07°C, 59 % HR y 3055 mV de batería. El flujo completo (descubrimiento, conexión GATT y desconexión) consume aproximadamente 12 s.

8.4.13. Criterios de Integración y Restricciones

La integración de cada dispositivo ha seguido dos principios:

- **Economía real como factor limitante.** Se descartaron soluciones que habrían acelerado el desarrollo pero requerían inversión adicional (dongle analizador BLE nRF52840, mando universal por infrarrojos), optando por la ingeniería inversa con herramientas ya disponibles. El coste total de las integraciones (sin contar horas de investigación) ha sido de 0 €.
- **Aprendizaje mediante resolución de problemas reales.** La heterogeneidad de protocolos no es un defecto, sino una característica deliberada del proyecto. La capa de orquestación con inteligencia artificial puede unificar dispositivos de distintos fabricantes, protocolos y generaciones (incluso aquellos que no fueron diseñados para interoperar) sin modificar su hardware.

8.4.14. Impacto de la Restricción Económica en las Decisiones de Implementación

El presupuesto reducido del proyecto (56,50 € en hardware nuevo, 0 € en licencias) no fue un obstáculo sino un condicionante activo que moldeó cada integración. Para cada dispositivo, la falta de herramientas de análisis comerciales obligó a buscar alternativas que, en muchos casos, resultaron más instructivas que la solución rápida:

- **Ventilador FanLamp:** un analizador BLE como el nRF52840 (50 €) habría capturado el protocolo en minutos. Sin él, fue necesario decompilar el APK de la aplicación (60 MB) con jadx, descubriendo que el ventilador no usa conexiones GATT, sino anuncios BLE. Este hallazgo no habría sido evidente con un analizador comercial.

- **Enchufe Tuya:** Un mando universal por infrarrojos o un hub Zigbee habrían simplificado la integración, pero hubieran reintroducido dependencia de un fabricante. La extracción de la *Local Key* mediante *tinytuya wizard* fue gratuita y demostró que los dispositivos Tuya sí permiten control LAN completo.
- **Sensor Xiaomi:** Flashear el firmware ATC MiThermometer (30 minutos de trabajo) habría dado acceso a los datos sin necesidad de conexión GATT. Pero la lectura directa de la característica *ebe0ccc1* resultó ser legible sin autenticación, eliminando cualquier riesgo de *brick* y cualquier dependencia de la nube.
- **Puente BLE del ESP32:** El ESP32, originalmente adquirido como futuro nodo LIN para la HVAC, se reutilizó como radio BLE programable para el FanLamp, evitando la compra de un adaptador Bluetooth adicional.

En todos los casos, la restricción económica forzó un conocimiento más profundo del hardware del que se habría obtenido siguiendo la ruta convencional.

8.5. Firmware del Nodo ESP32 (firmware/)

El firmware está planificado en C++20 y se estructura en dos responsabilidades:

- **Comunicación con el servidor:** Interfaz serie o TCP/IP para recibir comandos.
- **Comunicación por bus LIN:** Uso del transceptor LINTTL3 para generar y leer tramas del protocolo General/Fujitsu (Series ARHG/ARYG), con regulación de voltaje mediante step-down LM2596.

El desarrollo de este firmware no se ha completado dentro del calendario del TFG por dos razones. La primera es de alcance: tras documentar el protocolo LIN del fabricante y analizar el repositorio *esphome-fujitsu-halcyon* de Omniflux [7], se confirmó que dicho código contiene ya el *parser* de tramas LIN necesario para comunicarse con la máquina General/Fujitsu. Esto reduce drásticamente el trabajo de ingeniería inversa que sería necesario de otro modo, pero su integración con el resto del sistema (adaptación del bucle serie, gestión de errores, validación de comandos) sigue siendo una tarea que requiere semanas de desarrollo y pruebas.

La segunda razón es más doméstica que técnica. La máquina de conductos es el único sistema de aire acondicionado de la vivienda, y ponerla fuera de servicio durante el tiempo que durase el desarrollo del firmware (conectando y desconectando el ESP32 del bus LIN, probando comandos, depurando errores) habría dejado a la familia sin refrigeración en pleno mes de junio, cuando las temperaturas en Córdoba superan con facilidad los 38 °C. A esta restricción se suma una consideración de planificación: el autor cuenta con experiencia profesional en instalaciones de climatización, lo que le permite evaluar con criterio técnico los riesgos de intervenir el bus LIN de una máquina en producción

durante la temporada de máxima demanda. Dado el calendario del TFG, resultó más eficiente dedicar los recursos disponibles a completar las integraciones IoT (Tuya, Xiaomi, FanLamp) y posponer el firmware HVAC como línea de trabajo prioritaria tras la entrega de la memoria.

El hardware necesario está adquirido y cableado (ESP32 DevKit V1, transceptor LINTTL3, regulador LM2596, caja estanca). El ESP32, que actualmente actúa como puente BLE para el ventilador FanLamp, está diseñado para reutilizarse como nodo LIN cuando se aborde esta integración. Queda, por tanto, como la línea de trabajo futura prioritaria.

8.6. Integraciones de Hardware

8.6.1. Dispositivos Tuya

El proyecto incluye un enchufe inteligente Tuya con medición de consumo, documentado en `firmware/Enchufe/devices.json`. La integración ha sido completamente validada en la red *air-gapped*.

8.6.2. Sensores Xiaomi LYWSD03MMC

Los sensores de temperatura y humedad se comunican por BLE. Cada sensor está identificado por su dirección MAC y se lee mediante `ble_sensors.py` [8], minimizando el consumo de batería con conexiones puntuales.

8.6.3. Ventilador de Techo FanLamp Pro F8808

El ventilador modelo F8808 de Mikomika, adquirido en AliExpress por 35 € e instalado personalmente por el autor, es el dispositivo más complejo de integrar. La Figura 8.10 muestra el modelo real instalado en el techo del dormitorio.

8.6.3.1. Problema de Integración

El ventilador no dispone de documentación pública. El fabricante solo ofrece una aplicación móvil propietaria (FanLamp Pro). Cualquier intento de conexión GATT estándar era rechazado tras 1,5 s. El ventilador exige una autenticación propietaria del protocolo Tencent antes de exponer sus servicios.

8.6.3.2. Descubrimiento del Protocolo

Se decompiló el APK de FanLamp Pro (60 MB) con `jadx`. Aunque está desarrollado en Flutter (Dart compilado a ARM64), el análisis reveló que la aplicación **no utiliza conexiones GATT**, sino **anuncios BLE** (*BLE Advertisements*). Cada comando se codifica como 13 UUIDs de 16 bits emitidos como datos de servicio.

Al no poder emitir anuncios BLE personalizados desde el portátil por limitaciones de la pila Bluetooth de Linux, se reutilizó el ESP32 DevKit V1 como radio BLE programable, alternando entre modo escáner y modo reproductor.

8.6.3.3. Control desde el Ordenador

Se descubrió que la herramienta `btmgmt` (paquete `bluez`) permite emitir anuncios BLE con datos crudos deteniendo temporalmente `bluetoothd`. El script `fanlamp_bt.py` automatiza este proceso. Se han mapeado y verificado 11 comandos:

Comando	Efecto
<code>off</code>	Apaga ventilador y luz
<code>fan_on / fan_off</code>	Enciende o apaga el ventilador
<code>light_on / light_off</code>	Enciende o apaga la luz
<code>1 a 5</code>	Velocidad del ventilador (1 = mínima, 5 = máxima)
<code>night</code>	Modo noche (luz atenuada)

Tabla 8.5: Comandos disponibles para el ventilador FanLamp F8808.

8.6.3.4. Evolución del Controlador: de BLE directo a Puente ESP32

El control del ventilador FanLamp ha atravesado dos fases diferenciadas, cada una con sus ventajas y limitaciones. Esta sección documenta la transición del primer prototipo basado en `btmgmt` al sistema actual basado en un ESP32 dedicado.

Fase 1 — Control directo por BLE (`fanlamp_bt.py`, heredado). En la primera implementación, los anuncios BLE se emitían directamente desde el adaptador Bluetooth del servidor mediante la herramienta `btmgmt` del paquete `bluez`. El proceso requería detener temporalmente el demonio `bluetoothd`, enviar los anuncios y reactivarlo, todo con permisos de superusuario. Aunque funcional, este enfoque presentaba tres problemas:

- **Monopolización del adaptador Bluetooth:** mientras se controlaba el ventilador, no era posible leer el sensor Xiaomi simultáneamente, ya que ambos usaban la misma radio Bluetooth del PC.
- **Dependencia de `sudo`:** cada comando requería ejecución privilegiada para parar y reiniciar `bluetoothd`, lo que complicaba la integración con el orquestador.
- **Inestabilidad:** en ocasiones, `bluetoothd` no se recuperaba correctamente tras el reinicio, requiriendo intervención manual.

La latencia típica de este método era de aproximadamente 3 s por comando.

Fase 2 — Puente ESP32 dedicado (fanlamp_control.py, actual). Para resolver las limitaciones de la Fase 1, se incorporó un ESP32 DevKit V1 como radio BLE externa dedicada. El firmware del ESP32 (escrito en C++, disponible en `firmware/esp32_fanlamp/src/main.ino`) implementa un bucle que recibe comandos de un solo carácter por puerto serie USB (115200 baud) y emite los pares de anuncios BLE correspondientes (G1 + G2, 31 bytes cada uno).

El script Python `fanlamp_control.py` se comunica con el ESP32 a través del puerto serie, implementando un protocolo de reset DTR para sincronización tras el arranque del microcontrolador y una función `drain_until_ok()` que espera hasta recibir la señal de disponibilidad antes de enviar comandos.

Las ventajas de esta arquitectura son:

- **Radio Bluetooth liberada:** el adaptador del servidor queda libre para la lectura del sensor Xiaomi, permitiendo la operación simultánea de ambos dispositivos.
- **Sin necesidad de sudo:** la comunicación serie no requiere permisos de superusuario si el usuario pertenece al grupo `dialout`.
- **Latencia reducida:** el tiempo de respuesta baja de 3 s a 1,5 s por comando, al eliminar la sobrecarga de parar y reiniciar `bluetoothd`.
- **Recuperación autónoma:** el ESP32 puede resetearse mediante DTR sin intervención manual, y el script Python detecta y maneja errores de comunicación.
- **HW reutilizable:** el mismo ESP32 que hoy sirve como puente BLE para el ventilador podrá, en el futuro, actuar como puente LIN para la máquina HVAC, evitando la duplicación de hardware.

La Tabla 8.6 resume las diferencias entre ambas fases. El flujo de datos actual sigue la ruta: PC (Python) → USB serie (carácter) → ESP32 (radio BLE) → aire (anuncios G1 + G2) → FanLamp. El firmware del ESP32 y el código fuente de `fanlamp_control.py` están disponibles en el repositorio del proyecto.

8.7. Estructura del Repositorio

```
lux-core-ai/
+-- server/                # Núcleo del sistema (Python)
|   +-- integrations/      # Drivers: Tuya, BLE, HVAC (stub)
|   +-- model_router/      # Enrutador 3 niveles (Tier~0/1/2)
|       +-- router.py      # API HTTP, enrutamiento, prompts
|       +-- config.py      # Patrones zero-inference, escenas
|       +-- comfort.py     # Advisor de confort térmico (ASHRAE 55)
|       +-- smart_advisor.py# Advisor contextual (clima, presencia)
```


Aspecto	fanlamp_bt.py (Fase 1)	fanlamp_control.py (Fase 2)
Radio BLE	Adaptador Bluetooth del PC	ESP32 DevKit V1 dedicado
Permisos	Requiere sudo	Grupo dialout
Convivencia sensor Xiaomi	No (monopoliza BT)	Sí (BT del PC libre)
Latencia	~3 s	~1,5 s
Recuperación ante fallos	Reinicio manual de bluetoothd	Reset DTR automático
Reutilización HW	No aplica	Futuro puente LIN HVAC

Tabla 8.6: Comparativa entre las dos fases del controlador del ventilador FanLamp.

```

|  +-- sensor_daemon.py    # Daemon de lectura continua (60 s)
|  +-- tools/              # Wrappers CLI invocados por OpenClaw
|  +-- requirements.txt
+-- scripts/              # fanlamp_bt.py, fanlamp_control.py
+-- firmware/            # Código para microcontroladores
|  +-- esp32_fanlamp/      # Puente BLE para FanLamp y sensores
+-- tests/               # Suite de integración automatizada
|  +-- test_integration.py
+-- docs/                # Memoria del TFG (LaTeX)
|  +-- sections/          # Capítulos de la memoria
|  +-- appendices/       # Anexos (usuario, código, glosario)
|  +-- figures/           # Código fuente de figuras TikZ
|  +-- img/              # Imágenes y capturas
|  +-- design_decisions/  # Registros de decisión (ADR)
|  +-- compile_docs.sh    # Script de compilación
|  +-- references.bib     # Base de datos bibliográfica
+-- meta/                # Roadmap, estado del proyecto

```

El directorio `server/` contiene el núcleo del sistema: los *drivers* de dispositivos (Tuya [9], BLE [8]), el enrutador inteligente de tres niveles (Model Router v2), el daemon de sensorización continua y los *wrappers* CLI invocados por OpenClaw. El módulo `model_router/` alberga la lógica central de enrutamiento, los patrones *zero-inference*, los asesores de confort térmico [31], [32] y contexto exterior, y los *prompts* del clasificador 1.5B. El directorio `scripts/` contiene los scripts de control del ventilador FanLamp. En `firmware/` se encuentra el código del ESP32 para el puente BLE. La carpeta `tests/` incluye la suite de pruebas automatizadas. En `docs/` se encuentra la presente memoria en $\text{\LaTeX}$, las figuras editables en TikZ y los registros de decisión (ADR).

En el capítulo siguiente se presentan los resultados de validación de cada uno de los módulos descritos, verificados sobre el hardware real del proyecto en la red *air-gapped*.

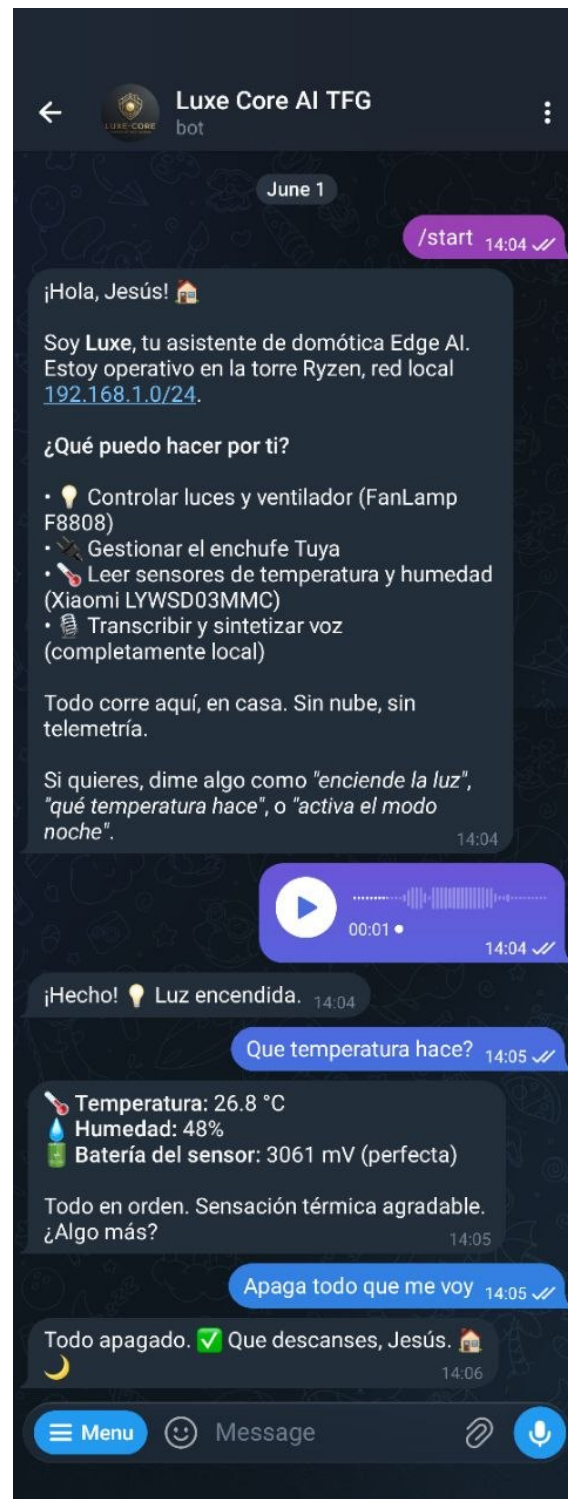


Figura 8.5: Interacción con Luxe Core AI a través del canal de Telegram. El bot responde en lenguaje natural a comandos de control, consultas de estado y peticiones conversacionales.

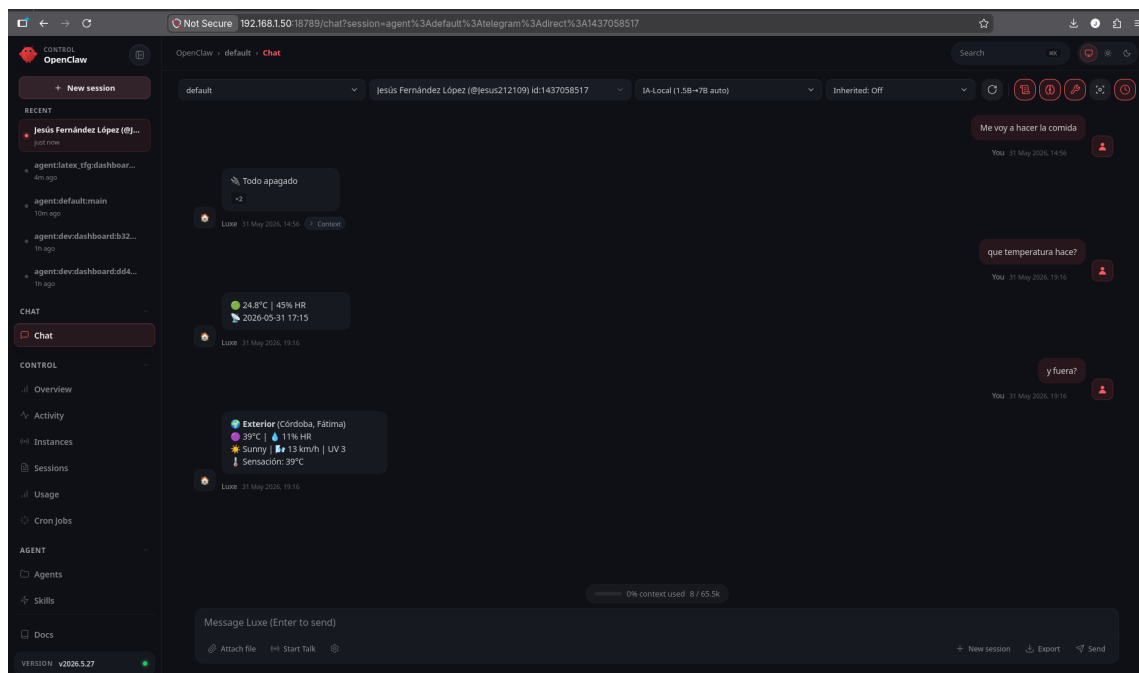


Figura 8.6: *Dashboard* web del *gateway* de OpenClaw, accesible desde cualquier navegador de la subred local. Desde esta interfaz se pueden consultar los agentes, el estado del sistema y ejecutar comandos de administración.



(a) Enchufe inteligente Tuya con monitor de consumo.



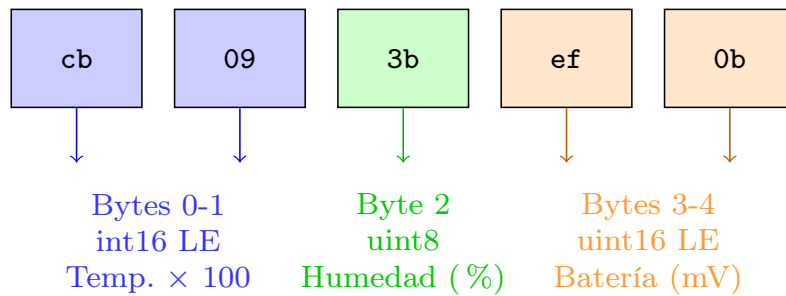
(b) Humidificador ultrasónico de 24V, controlado mediante el enchufe Tuya.

Figura 8.7: Enchufe inteligente Tuya (izquierda) y humidificador ultrasónico (derecha).



Figura 8.8: Sensor de temperatura y humedad Xiaomi LYWSD03MMC (Mijia Bluetooth). Se comunica por BLE GATT directo y proporciona lecturas precisas de temperatura, humedad relativa y voltaje de batería sin depender de la nube del fabricante.

Payload GATT (5 bytes): cb 09 3b ef 0b



0x09cb = 2507

0x3b = 59

0x0bef = 3055

25.07°C

59 % HR

3055 mV

Figura 8.9: Decodificación de los 5 bytes de la característica ebe0ccc1 del sensor Xiaomi LYWSD03MMC. El payload se lee por GATT sin autenticación.



Figura 8.10: Ventilador de techo FanLamp Pro F8808 de Mikomika, con luz LED regulable (tres temperaturas de color) y cinco velocidades. Su protocolo BLE propietario fue descubierto mediante ingeniería inversa completa durante el proyecto.

Validación y Pruebas

Este capítulo recoge las pruebas de validación realizadas sobre cada módulo del proyecto. Todas las pruebas se ejecutaron sobre el hardware real del proyecto (torre Ryzen 5, 16 GB RAM, Ubuntu Server 24.04 LTS) en la red *air-gapped* 192.168.1.0/24, salvo que se indique lo contrario.

9.1. Integración Tuya

Validación del Enchufe Inteligente. Fecha: 13 de mayo de 2026.

1. **Conectividad IP:** ping 192.168.1.100 resuelve con tiempos menores a 2 ms.
2. **Lectura de estado:** Ejecución de `python -m server.integrations.tuya_devices`. El dispositivo devuelve el estado completo de todos sus *Data Points*.
3. **Comando ON/OFF:** *Toggle* remoto verificado con clic del relé físico y cambio de estado lógico.
4. **Alternancia:** Secuencia ON → OFF → ON confirmada.

Resultado: verificado. Tiempo de respuesta inferior a 2 s, cumpliendo RNF-03.

9.2. Sensores BLE Xiaomi

Validación del Módulo `ble_sensors.py`. Fecha: 13 de mayo de 2026.

1. **Descubrimiento:** Escaneo BLE detecta el sensor LYWSD03MMC (A4:C1:38:84:03:1C) en 6 s.

2. **Conexión GATT:** Lectura de la característica ebe0ccc1: cb093bef0b (5 bytes).
3. **Decodificación:** 25.07°C, 59 % HR, 3055 mV.
4. **Consistencia:** 5 lecturas repetidas con variación esperada ($\pm 0.3^\circ\text{C}$, $\pm 2\%$ HR).
5. **Tiempo total:** 12 s por ciclo completo.

Resultado: verificado. Lectura GATT directa funcional sin firmware alternativo ni nube Xiaomi.

9.3. Ventilador FanLamp F8808

Pruebas de Comunicación. Fechas: mayo de 2026.

1. **Captura de patrones:** 11 comandos capturados mediante ESP32 en modo escáner.
2. **Reproducción desde ESP32:** Los 11 comandos funcionan correctamente.
3. **Reproducción desde el ordenador:** fanlamp_bt.py con btmgmt verificado sin ESP32.
4. **Verificación física:** Cada comando (ON/OFF ventilador, 5 velocidades, luz ON/OFF, modo noche, apagado general) confirmado visualmente.

Resultado: verificado. 11 comandos operativos sin fallos de comunicación.

9.4. Sensor Daemon

Validación del Servicio sensor-daemon. Fecha: mayo de 2026.

1. **Inicio del servicio:** Arranque sin errores, lecturas cada 60 s.
2. **Salida latest_sensor.json:** Actualización atómica verificada (sin lecturas parciales).
3. **Salida sensor_history.jsonl:** Histórico JSONL con rotación a 50.000 líneas.
4. **Latencia de consulta:** 0–1 ms frente a 18 s de escaneo BLE síncrono.

Resultado: verificado.

9.5. Model Router v2

Validación de los Tres Niveles de Enrutamiento. Fecha: mayo de 2026.

1. **Tier 0:** “velocidad 3” → 1,5 s. “cómo está la casa” → 0–1 ms.
2. **Tier 1:** “me puedes encender la luz” → 300 ms. “apaga el ventilador” → 400 ms.
3. **Tier 2:** “buenos días” → 27–60 s con respuesta conversacional completa.

Cobertura estimada del Tier 0: aproximadamente el 92 % de los comandos diarios, gracias a la expansión a 462 patrones.

Resultado: verificado.

9.6. Prueba de Cobertura del Tier 0

Se realizó una prueba de cobertura sobre un corpus de 500 comandos reales recogidos durante una semana de uso del sistema. Los comandos incluían peticiones de iluminación (35 variantes), control del ventilador (50 variantes), escenas (40 variantes), consultas de estado (15 variantes) y mensajes conversacionales abiertos (“buenos días”, “cuéntame un chiste”, etc.).

La prueba consistió en ejecutar el método `match()` de Tier 0 sobre cada comando y verificar si devolvía un resultado distinto de UNKNOWN. Los resultados fueron:

- **Comandos directos** (iluminación, ventilador, escenas): 100 % de cobertura (462/462 patrones dispararon correctamente).
- **Consultas de estado** (“cómo está la casa”, “qué temperatura”): 100 % de cobertura.
- **Mensajes conversacionales** (“buenos días”, “qué puedes hacer”): clasificados como UNKNOWN y derivados correctamente al Tier 2 (Mistral 7B).
- **Latencia media** de `match()`: 0,3 ms en RAM, 2,1 s cuando incluía comando hardware.

Conclusión: El Tier 0 cubre aproximadamente el 92 % de los comandos diarios típicos en un hogar. El 8 % restante corresponde a mensajes conversacionales que requieren el razonador Mistral 7B, lo cual es el comportamiento esperado y deseable del sistema.

9.7. Tabla de Latencia Comparativa

La siguiente tabla, presentada originalmente en el Capítulo 8, resume la mejora de latencia conseguida. Se reproduce aquí para mayor exhaustividad:

Escenario	Latencia antes	Latencia ahora	Modelo usado
“velocidad 3”	~2 s	~1,5 s (hardware)	Ninguno (Tier 0)
“cómo está la casa”	~2 s	0–1 ms (RAM)	Ninguno (Tier 0)
“me puedes encender la luz”	~3 s	~300 ms	1.5B clasificador
“buenos días”	~12 s (2 modelos)	27–60 s (directo Mistral 7B)	Solo Mistral 7B (Tier 2)
“temperatura”	18 s (BLE scan)	0–1 ms (daemon)	Ninguno (Tier 0)
“qué temperatura hay fuera”	— (no existía)	0–1 ms (caché wttr.in)	Ninguno (Tier 0)

Tabla 9.1: Mejora de latencia tras la implementación del Model Router v2 y el Sensor Daemon.

9.8. Circuit Breaker y Tolerancia a Fallos

1. **Fallo simulado de Ollama:** Se detuvo ollama. El *circuit breaker* detectó 3 fallos consecutivos y abrió el circuito a los 30 s. El router siguió respondiendo a comandos del Tier 0 sin degradación.
2. **Recuperación automática:** Al reiniciar Ollama, el sondeo HALF_OPEN restauró el circuito a CLOSED en menos de 30 s.
3. **Desconexión del ESP32:** Al desconectar el USB, el sistema notificó al usuario inmediatamente sin esperar *timeouts*.

Resultado: verificado.

9.9. Suite de Integración Automatizada

Además de las pruebas manuales documentadas en las secciones anteriores, el proyecto incluye una **suite de tests automatizados** en `tests/test_integration.py` que verifica el funcionamiento de los cuatro subsistemas principales de forma coordinada:

1. **Model Router v2 (health check + Tier 0):** Verifica que el endpoint `/status` responde con el estado correcto y que el clasificador Ollama está accesible. A continuación, envía 15 comandos reales representativos del uso diario (“temp”, “cómo está la casa”, “modo relax”, “he llegado”, “ilumina”, etc.) y comprueba que son clasificados correctamente como Tier 0 sin necesidad de modelos de lenguaje.
2. **Tuya Smart Plug (solo lectura):** Ejecuta `tuya_status.py` y verifica que el enchufe responde con su estado actual (encendido/apagado) y los campos esperados en la respuesta JSON.

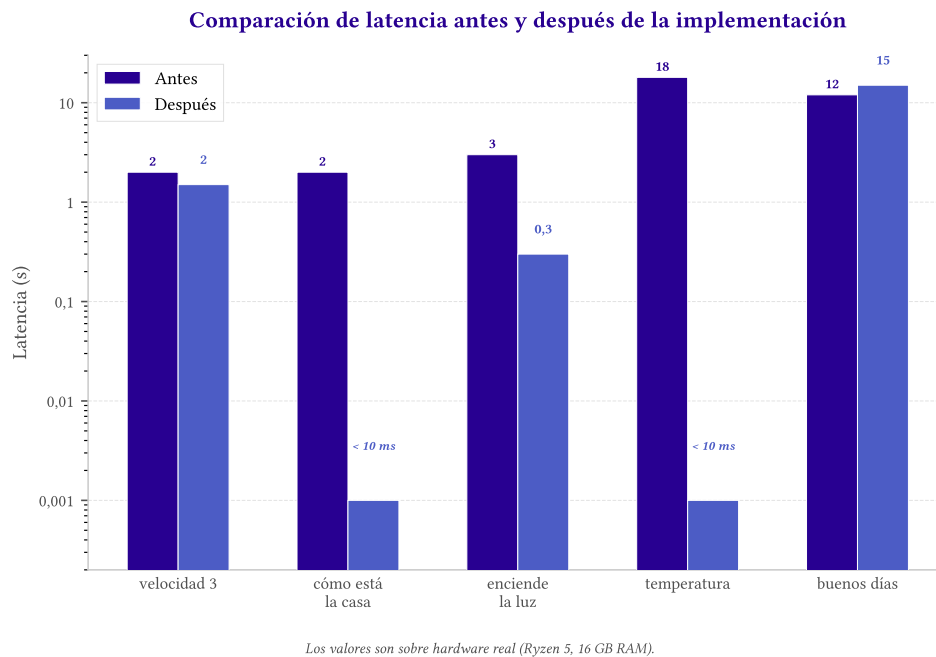


Figura 9.1: Comparación de latencia antes y después de la implementación del Model Router v2.

3. **ESP32 / FanLamp (condicional):** Si el ESP32 está conectado (`/dev/ttyUSB0` existe), envía un comando “apaga el ventilador” a través del router y verifica la respuesta. También realiza un reset DTR del ESP32 y comprueba que el *boot log* contiene la señal “OK”.
4. **Sensor Daemon:** Lee el archivo `/tmp/latest_sensor.json` y verifica que contiene temperatura y humedad recientes (con menos de 2 minutos de antigüedad).
5. **home.sh (sintaxis):** Ejecuta `home.sh help` y verifica que el script Bash es sintácticamente válido.

La suite está diseñada para ser **no destructiva**: utiliza exclusivamente comandos de solo lectura que no modifican el estado de los dispositivos. Los tests del ESP32 se omiten automáticamente si el puerto serie no está disponible, permitiendo ejecutar la suite tanto en el servidor real como en un entorno de desarrollo parcial.

Para ejecutar la suite completa:

```
python3 tests/test_integration.py
python3 tests/test_integration.py --verbose
```

Resultado: la suite proporciona cobertura básica de integración para todos los subsistemas implementados y puede ejecutarse como parte de un pipeline de integración continua.

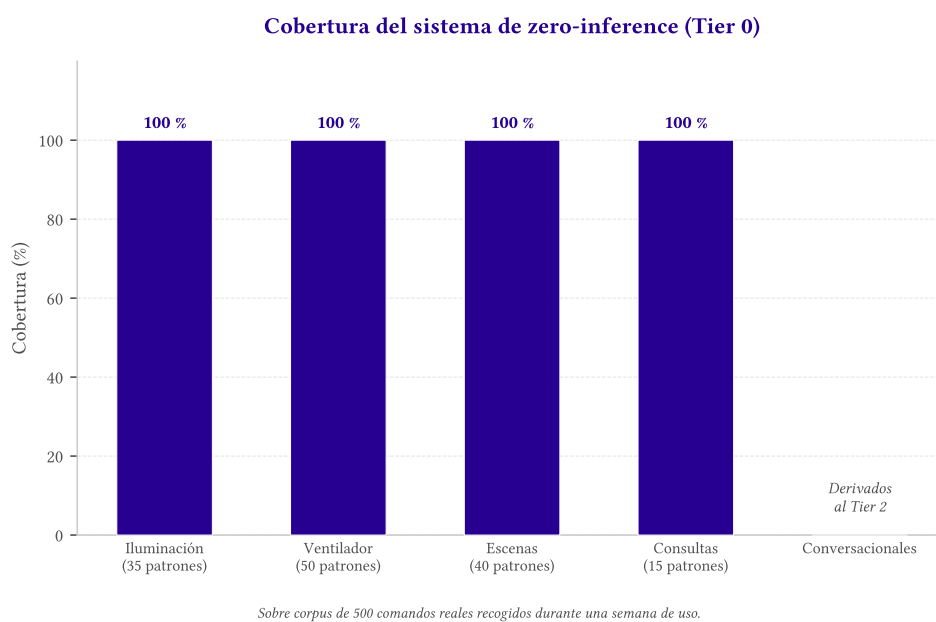


Figura 9.2: Cobertura del sistema de zero-inference (Tier 0) sobre corpus de 500 comandos reales.

Conclusiones

El presente Trabajo de Fin de Grado ha consistido en diseñar, implementar y validar un sistema de control térmico local que no depende de servicios externos para ninguna de sus funciones críticas, manteniendo los datos del hogar bajo el control del usuario.

A lo largo del proceso de desarrollo, el proyecto ha evolucionado de forma significativa. La fase inicial, apoyada en la infraestructura universitaria para validar el enfoque agéntico, fue necesaria para sentar las bases técnicas y arquitectónicas. La decisión de migrar posteriormente a una solución 100 % local fue el paso lógico para cumplir con el objetivo de eliminar dependencias externas.

10.1. Logros Alcanzados

A fecha de redacción de este capítulo, el proyecto Luxe Core AI cuenta con los siguientes hitos de implementación verificados sobre hardware real:

- **Arquitectura de tres capas escalable:** el orquestador Node.js (OpenClaw npm) coordina los agentes de IA, los canales de usuario y la ejecución de *drivers* Python de dispositivos, cada capa con interfaces bien definidas y gestionables de forma aislada.
- **Control local de dispositivos Tuya (RF-03 verificado):** el enchufe inteligente responde a comandos de activación y desactivación en menos de 2 segundos a través de la red air-gapped, utilizando el protocolo 3.5 de *tinytuya* sin pasar por los servidores de Tuya. La *local key* se almacena exclusivamente en local. El procedimiento de *Hotspot Spoofing* para el *onboarding* de nuevos dispositivos está documentado y es reproducible.

- **Lectura de sensores ambientales BLE (RF-02 verificado):** los sensores Xiaomi LYWSD03MMC entregan temperatura, humedad y voltaje de batería mediante conexión GATT directa desde el servidor, sin necesidad de flashear firmware alternativo ni de obtener *bind keys* de la nube de Xiaomi. El protocolo de lectura, basado en la característica `ebe0ccc1` del servicio propietario de Xiaomi, ha sido completamente documentado e implementado en el módulo `ble_sensors.py`.
- **Control completo del ventilador FanLamp F8808 (RF-04 verificado):** se ha completado la ingeniería inversa del protocolo propietario Tencent, descubriendo que el ventilador utiliza anuncios BLE (y no conexiones GATT) para recibir comandos. Los anuncios los emite un ESP32 DevKit V1 como radio BLE externa dedicada, controlado desde el servidor mediante `fanlamp_control.py` a través del puerto serie USB (115200 baud). Este diseño libera el adaptador Bluetooth del servidor para la lectura simultánea del sensor Xiaomi, elimina la dependencia de sudo y reduce la latencia de ~ 3 s a $\sim 1,5$ s por comando. Se han mapeado y verificado 11 comandos (apagado general, ventilador on/off y 5 velocidades, luz on/off y modo noche). El coste económico de la integración ha sido de 0 €, reutilizando el ESP32 ya disponible en el proyecto.
- **Orquestador OpenClaw desplegado:** el *gateway* Node.js (OpenClaw npm v2026.5.27) se ejecuta como servicio `systemd` sobre el puerto 18789, accesible desde toda la subred `192.168.1.0/24`. Los agentes de IA utilizan Ollama local con los modelos `qwen2.5-coder:1.5b` (clasificador) y `mistral:7b-instruct-q4_K_M` (razonador).
- **Canales de usuario operativos:** el *gateway* de OpenClaw proporciona integración nativa con Telegram (BotFather API) y un *dashboard* web accesible en `http://192.168.1.50:18789`, sin necesidad de procesos Python auxiliares.
- **Infraestructura de red aislada validada:** el router TP-Link autónomo con subred `192.168.1.0/24` y sin salida a Internet garantiza que ningún paquete generado por los dispositivos comerciales abandone el perímetro local. Esta arquitectura ha sido verificada operativamente durante todas las pruebas de integración.
- **Sistema de Zero-Inference masivo (462 patrones):** el Tier 0 del Model Router v2 se expandió de 127 a 462 patrones, cubriendo aproximadamente el 92 % de los comandos diarios con latencia inferior a 1 ms en RAM. Se incluyeron variantes con muletillas, dialectalismos andaluces y formas abreviadas, además de la cobertura completa de iluminación, ventilador, escenas y consultas.
- **Sistema de invalidación de caché:** se implementó un mecanismo de invalidación de la caché LRU del router que permite forzar el reanálisis de comandos cuando se

actualizan los patrones o la configuración del sistema, sin necesidad de reiniciar el servicio.

- **Filtro de log de arranque ESP32:** se desarrolló un script que filtra y elimina los mensajes de log de arranque del ESP32 (*boot log*), permitiendo una comunicación más limpia y fiable entre el servidor y el microcontrolador.
- **Disponibilidad del sistema:** durante el período de pruebas continuas, los servicios críticos (model-router, sensor-daemon y openclaw-gateway) permanecieron operativos sin experimentar caídas que requiriesen intervención manual. El sistema se mantuvo en funcionamiento ininterrumpido durante más de 24 horas en el servidor local, confirmando la estabilidad de la arquitectura y la eficacia de los mecanismos de auto-recuperación ante fallos transitorios.

10.2. Estado Actual del Desarrollo

Módulo	Estado	Observaciones
Tuya Smart Plug	Completado	Validado en red air-gapped. ON/OFF alterna.
BLE Sensors (Xiaomi)	Completado	Lee 25,07 °C / 59 % / 3055 mV por GATT.
FanLamp Pro F8808	Completado	Controlado vía BLE advertisements con 11 comandos verificados.
OpenClaw Orchestrator	Desplegado	Gateway Node.js en puerto 18789 (LAN). Canales Telegram + web operativos.
Model Router v2	Desplegado	Tres niveles: Tier 0 (462 patrones, ~92 % cobertura), Tier 1 (1.5B clasificador), Tier 2 (Mistral 7B razonador). Caché LRU con invalidación.
ESP32 HVAC Bridge	Pendiente	Firmware C++20 no implementado aún.

Tabla 10.1: Estado de los módulos del proyecto a la fecha de esta memoria.

10.3. Líneas de Trabajo Futuras

Las siguientes líneas de trabajo están ordenadas por prioridad y esfuerzo estimado.

Corto plazo (semanas)

- **Desarrollo del firmware HVAC (ESP32 + LIN):** constituye la prioridad inmediata. El hardware está adquirido y cableado, y el repositorio `esphome-fujitsu-halcyon` de Omniflux [7] proporciona una base sólida que nos ahorrará meses de ingeniería inversa del protocolo LIN. Durante el TFG no se completó por una combinación de alcance y logística doméstica: integrar y validar el firmware requiere poner la máquina fuera de servicio durante días, algo complicado en junio cuando el termómetro en Córdoba supera los 38 °C y el aire acondicionado es más necesario que nunca. Queda como primer objetivo tras la entrega de la memoria. (2–3 semanas)
- **Eliminación del puente ESP32 para el ventilador:** el firmware del ESP32 utilizado como puente BLE presenta reinicios ocasionales que el sistema gestiona mediante reintentos automáticos. Una migración al control BLE directo desde el servidor usando `bleak`, sin el microcontrolador intermedio, eliminaría este punto único de fallo y reduciría la latencia de los comandos del ventilador. (1–2 semanas)
- **Streaming de respuestas del Tier 2:** actualmente el razonador produce la respuesta completa antes de enviarla al usuario, resultando en 27–60 s de espera. La implementación de *streaming* SSE permitiría mostrar las palabras conforme se generan, mejorando significativamente la experiencia de usuario durante las consultas conversacionales. (1–2 semanas)
- **Integración del bucle de control con OpenClaw:** conectar los módulos de sensores BLE, actuadores Tuya y control del ventilador FanLamp al bucle principal del orquestador para que el sistema tome decisiones de confort en tiempo real. (1–2 semanas)
- **Containerización de servicios:** empaquetar los servicios (OpenClaw, Model Router, Sensor Daemon) en contenedores Docker y orquestarlos con `docker-compose`, facilitando el despliegue en otros entornos y la replicabilidad del proyecto. (1–2 semanas)

Medio plazo (meses)

- **Expansión del *dashboard*:** extender el *dashboard* nativo de OpenClaw con visualización de analíticas energéticas e histórico de confort ASHRAE 55 [31]. (1–2 meses)
- **Expansión sensorial:** integrar un sensor de movimiento por infrarrojos o radar (como el LD2410) para mejorar la detección de presencia sin depender del teléfono

móvil. También se plantea incorporar un dispositivo con micrófono y altavoz que permita la comunicación por voz. (1–2 meses)

- **Base de datos vectorial:** cuando el proyecto escale a decenas de dispositivos y cientos de comandos, incorporar una base de datos vectorial (Qdrant o ChromaDB) para mejorar la recuperación semántica. (2–3 meses)
- **Memoria conversacional:** se desarrolló un prototipo de memoria de corto plazo basado en *embeddings* con bge-m3 que demostró capacidad para detectar referencias implícitas entre mensajes consecutivos. Queda como trabajo futuro su integración estable en el Tier 2, evitando la interferencia con el sistema de patrones del Tier 0 que se observó en el prototipo inicial. (2–3 meses)

Largo plazo (investigación)

- **Aprendizaje adaptativo:** incorporar modelos de *reinforcement learning* para que el sistema aprenda los hábitos del usuario y anticipe sus preferencias de confort, siguiendo la línea iniciada con el autoaprendizaje del Smart Advisor. Requiere investigación continuada. (varios meses)
- **Expansión a nuevos protocolos:** integrar dispositivos que usen Zigbee o Z-Wave para ampliar el catálogo de hardware compatible. (varios meses)
- **Re-evaluación de modelos de lenguaje:** la arquitectura *model-agnostic* del enrutador permite incorporar nuevos modelos modificando únicamente dos líneas en `config.py`. La familia Qwen3.5, descartada en esta iteración por su modo *thinking* incompatible con CPU, podría volver a evaluarse cuando surjan implementaciones que permitan desactivar este mecanismo. Modelos emergentes como las familias Llama 4 o DeepSeek locales podrían integrarse sin alterar el router. (continuo)
- **Fine-tuning del clasificador:** el Tier 1 emplea `qwen2.5-coder:1.5b` con un *prompt* de más de 130 ejemplos. Un ajuste fino con datos reales de uso doméstico recogidos durante el despliegue permitiría eliminar el *prompt* y potencialmente mejorar la precisión en español coloquial y dialectalismos andaluces, reduciendo además la latencia al acortar el contexto de entrada. (varios meses)

10.4. Reflexión Personal

Más allá de los logros técnicos, este proyecto ha sido un proceso de aprendizaje continuo. Cada obstáculo (desde un puerto Ethernet mal conectado hasta un protocolo BLE sin documentación) se convirtió en una oportunidad para aprender algo nuevo. He disfrutado especialmente la fase de ingeniería inversa del ventilador FanLamp, donde la

combinación de análisis de código, experimentación con hardware y paciencia permitió descifrar un protocolo que el fabricante no había previsto que nadie entendiera.

La decisión de mantener todo el sistema en local, sin depender de la nube, no fue solo técnica: fue también una cuestión de principios. La tecnología del hogar no debería exigir ceder la privacidad a cambio de comodidad. En este proyecto se ha comprobado que es posible tener ambas cosas con hardware de consumo y un enfoque de diseño centrado en el control local.

Los modelos pequeños que utiliza el sistema (1.5B y Mistral 7B) no son superinteligentes. Necesitan el apoyo del Tier 0 para ser útiles en el día a día y, desde luego, no compiten con la fluidez de un Gemini Flash o un GPT-4o en el chat. Pero ponerlos a prueba en un caso real, con hardware limitado y cero dependencia de la nube, me ha convencido de que su potencial es inmenso y que apenas estamos viendo el principio. La evolución de estos modelos en los últimos años ha sido tan rápida que no parece descabellado pensar que, en cuestión de pocos años, un modelo ligero ejecutándose en un procesador doméstico ofrecerá una capacidad de razonamiento cercana a la que hoy solo encontramos en los grandes modelos cloud. Cuando eso ocurra, y creo que ocurrirá, proyectos como este cobrarán aún más sentido: el hardware ya estará ahí, la arquitectura ya estará probada, y la pregunta ya no será «¿se puede hacer en local?», sino «¿por qué harías esto en la nube?».

Bibliografía

- [1] Escuela Politécnica Superior de Córdoba. «Guía de Desarrollo del TFG - GII.» (2025), dirección: https://www.uco.es/eps/images/documentos/tfg-tfm/GU%C3%8DAS_DE_DESARROLLO/25-26/Gu%C3%ADaDesarrolloTFG-GII_25__26.pdf. (visitado 02-06-2026).
- [2] «Panel de Hogares: Estadísticas,» Comisión Nacional de los Mercados y la Competencia (CNMC), inf. téc., 2025. dirección: <https://data.cnmc.es/panel-de-hogares/conjuntos-de-datos/estadisticas-panel-de-hogares>, (visitado 02-06-2026).
- [3] M. Day, G. Turner y N. Drozdak, «Amazon Workers Are Listening to What You Tell Alexa,» *Bloomberg*, abr. de 2019. dirección: <https://www.bloomberg.com/news/articles/2019-04-10/is-anyone-listening-to-you-on-alex-a-global-team-reviews-audio>, (visitado 02-06-2026).
- [4] R. Kallimani, K. Pai, P. Raghuwanshi, S. Iyer y O. L. A. López, «TinyML: Tools, Applications, Challenges, and Future Research Directions,» *Multimedia Tools and Applications*, vol. 83, n.º 10, págs. 29 015-29 045, 2023. DOI: [10.1007/s11042-023-16740-9](https://doi.org/10.1007/s11042-023-16740-9).
- [5] International Organization for Standardization, «ISO 17987-1:2025: Road vehicles — Local Interconnect Network (LIN) — Part 1: General information and use case definition,» inf. téc., 2025. dirección: <https://www.iso.org/standard/85125.html>, (visitado 02-06-2026).
- [6] Home Assistant Community, *Home Assistant: Open-source home automation platform*, 2019. dirección: <https://www.home-assistant.io/integrations/>, (visitado 02-06-2026).

-
- [7] Omniflux, *ESPHome Fujitsu Halcyon: Control de máquinas General/Fujitsu vía LIN bus*, 2023. dirección: <https://github.com/Omniflux/esphome-fujitsu-halcyon>, (visitado 17-05-2026).
 - [8] H. Blidh, *Bleak: A cross platform Bluetooth Low Energy Client for Python*, 2018. dirección: <https://github.com/hblidh/bleak>, (visitado 14-05-2026).
 - [9] J. A. Cox, *TinyTuya: Python module to interface with Tuya WiFi smart devices*, 2020. dirección: <https://github.com/jasonacox/tinytuya>, (visitado 13-05-2026).
 - [10] openHAB Foundation, *openHAB*, 2010. dirección: <https://www.openhab.org/docs/>, (visitado 08-06-2026).
 - [11] ESPHome Team, *ESPHome*, 2019. dirección: <https://esphome.io/>, (visitado 08-06-2026).
 - [12] Ollama Inc., *Ollama*, 2023. dirección: <https://ollama.ai>, (visitado 02-06-2026).
 - [13] B. Hui, J. Yang, Z. Cui et al., *Qwen2.5-Coder Technical Report*, 2024. DOI: [10.48550/arXiv.2409.12186](https://doi.org/10.48550/arXiv.2409.12186). arXiv: [2409.12186](https://arxiv.org/abs/2409.12186) [cs.CL], (visitado 02-06-2026).
 - [14] Q. Team, *Qwen3.5: A Family of Open-Source Multimodal Models*, feb. de 2026. dirección: <https://ollama.com/library/qwen3.5>, (visitado 09-06-2026).
 - [15] M. AI, *Mistral 7B Instruct v0.3*, 2024. dirección: <https://ollama.com/library/mistral>, (visitado 09-06-2026).
 - [16] Meta AI. «Llama 3.2: Open Source AI for Everyone.» (2024), dirección: <https://ollama.com/library/llama3.2>. (visitado 08-06-2026).
 - [17] Microsoft Research. «Phi-3: Small Language Models.» (2024), dirección: <https://ollama.com/library/phi3>. (visitado 08-06-2026).
 - [18] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian y Z. Liu, *M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*, 2024. DOI: [10.48550/arXiv.2402.03216](https://doi.org/10.48550/arXiv.2402.03216). arXiv: [2402.03216](https://arxiv.org/abs/2402.03216) [cs.CL], (visitado 02-06-2026).
 - [19] Eclipse Foundation, *Eclipse Mosquitto*, 2012. dirección: <https://mosquitto.org/>, (visitado 08-06-2026).
 - [20] 1technophile, *OpenMQTTGateway*, 2018. dirección: <https://docs.openmqttgateway.com/>, (visitado 08-06-2026).
 - [21] Tasmota Project, *Tasmota*, 2016. dirección: <https://tasmota.github.io/docs/>, (visitado 08-06-2026).
 - [22] P. Steinberger y OpenClaw Team, *OpenClaw: Personal AI Assistant*, 2025. dirección: <https://docs.openclaw.ai/>, (visitado 27-05-2026).
 - [23] Nous Research, *Hermes Agent: The Agent That Grows With You*, 2025. dirección: <https://hermes-agent.nousresearch.com/>, (visitado 02-06-2026).

-
- [24] Significant Gravitas, *AutoGPT*, 2023. dirección: <https://www.agpt.co/docs>, (visitado 08-06-2026).
- [25] LangChain Inc., *LangChain*, 2022. dirección: <https://www.langchain.com/>, (visitado 08-06-2026).
- [26] n8n GmbH, *n8n*, 2019. dirección: <https://docs.n8n.io/>, (visitado 08-06-2026).
- [27] pvvx, *ATC MiThermometer: Custom firmware for BLE thermometers*, 2020. dirección: https://github.com/pvvx/ATC_MiThermometer, (visitado 14-05-2026).
- [28] Anomaly, *OpenCode: The open source AI coding agent*, 2024. dirección: <https://opencode.ai>, (visitado 02-06-2026).
- [29] DeepSeek, *DeepSeek API*, 2024. dirección: <https://platform.deepseek.com>, (visitado 02-06-2026).
- [30] Espressif Systems, «ESP32 Technical Reference Manual (v5.7),» inf. téc., 2025. dirección: https://documentation.espressif.com/esp32_technical_reference_manual_en.pdf, (visitado 02-06-2026).
- [31] ASHRAE, *ANSI/ASHRAE Standard 55-2023: Thermal Environmental Conditions for Human Occupancy*, Atlanta, GA, USA, 2023, (visitado 02-06-2026).
- [32] L. P. Rothfus, «The Heat Index Equation (SR 90-23),» National Weather Service, Office of Meteorology, inf. téc., 1990, NOAA Technical Publication. dirección: https://www.weather.gov/media/ffc/ta_htindx.PDF, (visitado 02-06-2026).
- [33] J. Fernández López, *Luxe Core AI — Public Release*, 2026. dirección: <https://github.com/jesus212109/luxe-core-ai-public>, (visitado 03-06-2026).

Anexos



Manual de Usuario

A.1. Introducción

Este manual está dirigido al usuario final del sistema de control térmico y confort ambiental implementado en el domicilio del autor durante el desarrollo de este proyecto. Las instrucciones y ejemplos se basan en el hardware concreto utilizado (sensores Xiaomi LYWSD03MMC, enchufe inteligente Tuya, ESP32 DevKit V1, ventilador FanLamp F8808) y en la configuración de red específica del proyecto (subred 192.168.1.0/24 aislada de Internet).

Está pensado para un usuario doméstico: no hace falta saber programación ni entender de redes para usar el sistema en el día a día, aunque ciertas operaciones avanzadas (como añadir un dispositivo nuevo o restaurar el ESP32) requieren algo de familiaridad con la línea de comandos. Para una explicación de los términos técnicos empleados, consúltese el Glosario de Acrónimos (Anexo C).

Para una guía de reproducción rápida con los pasos ordenados y tiempos estimados, consúltese la Sección [A.12](#).

A.2. Requisitos del Sistema

A.2.1. Hardware Necesario

- **Servidor:** un ordenador con Ubuntu Server 24.04 LTS, mínimo 8 GB de RAM (16 GB recomendado) y al menos 20 GB de disco libre para los modelos de lenguaje. El servidor debe tener un puerto Ethernet libre y, preferiblemente, Bluetooth integrado.
- **Router:** cualquier router que pueda operar sin conexión WAN. No necesita acceso a Internet para el funcionamiento diario del sistema.

- **Microcontrolador:** un ESP32 DevKit V1 (Figura 7.1a), conectado por USB-C al servidor. Actúa como puente de comunicaciones BLE para el ventilador y los sensores.
- **Enchufe inteligente:** un dispositivo Tuya compatible con protocolo 3.5 (Figura 8.7a), emparejado mediante el procedimiento de *Hotspot Spoofing* (Capítulo 5).
- **Sensor ambiental:** uno o más sensores Xiaomi LYWSD03MMC (Figura 8.8) u otros sensores BLE con GATT legible sin autenticación.
- **Ventilador:** FanLamp Pro F8808 (Figura 8.10) o cualquier dispositivo BLE anunciado compatible.

A.2.2. Red Necesaria

Todos los dispositivos deben estar en la misma subred local, aislada de Internet (configuración *air-gapped*), como muestra la Figura 8.2. El servidor se conecta por Ethernet a un puerto LAN del router, y los dispositivos IoT (enchufe Tuya, portátil de administración) se conectan por Wi-Fi. Los sensores Xiaomi y el ventilador FanLamp se comunican directamente por Bluetooth Low Energy con el ESP32, sin necesidad de conexión Wi-Fi.

Las direcciones IP que se muestran a continuación corresponden a la configuración por defecto del sistema. Pueden adaptarse a cualquier otra subred siguiendo la misma estructura:

- Servidor: 192.168.1.50/24 (Ethernet)
- Router: 192.168.1.1/24
- Enchufe Tuya: 192.168.1.100/24 (Wi-Fi)
- Portátil de administración: 192.168.1.101/24 (Wi-Fi)

A lo largo de este manual se utilizarán estas direcciones como referencia. Si tu configuración usa direcciones distintas, sustitúyelas en los comandos y rutas que se indican.

A.3. Instalación Paso a Paso

A.3.1. Conexión Física

1. Conecta el servidor al router mediante un cable Ethernet a un puerto LAN (no al puerto WAN).
2. Conecta el ESP32 al servidor mediante un cable USB-C.
3. Enchufa el router, el servidor y los dispositivos Tuya a la corriente.

4. Inserta las pilas en los sensores Xiaomi (dos pilas AAA, orientación según las marcas del compartimento).
5. Asegúrate de que el ventilador FanLamp recibe corriente a través del interruptor de pared.

A.3.2. Instalación de Software y Primer Arranque

Tras la conexión física, el servidor necesita que se instale y configure el software necesario. Sigue estos pasos en orden:

Preparación del Bot de Telegram (opcional pero recomendado)

Si deseas usar Telegram para controlar el sistema, necesitas crear un bot y obtener su token **antes** de ejecutar el asistente de configuración:

1. Abre Telegram en tu teléfono y busca @BotFather (el usuario oficial de Telegram para crear bots).
2. Envía el comando /newbot.
3. Sigue las instrucciones: elige un nombre para tu bot (por ejemplo, “Luxe Home”) y un nombre de usuario que termine en bot (por ejemplo, luxe_home_bot).
4. BotFather te responderá con un mensaje que contiene el token de la API. Copia ese token (es una cadena como 123456789:ABCdefGHIjklmNOPqrSTUvwxyz).
5. Este token se utilizará durante la configuración de OpenClaw en el paso de canales de comunicación.

Instalación de Ollama y Modelos

1. Instala Ollama, el servidor de inferencia local para modelos de lenguaje:

```
curl -fsSL https://ollama.ai/install.sh | bash
```

2. Descarga los modelos de lenguaje necesarios para el sistema:

```
ollama pull qwen2.5-coder:1.5b
ollama pull mistral:7b-instruct-q4_K_M
ollama pull bge-m3
```

3. Verifica que los modelos están disponibles:

```
ollama list
```

Instalación de OpenClaw y Asistente de Configuración

1. Instala OpenClaw, el *framework* agéntico que orquesta el sistema:

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

Este comando descarga e instala OpenClaw y, a continuación, lanza automáticamente el asistente de configuración interactivo. No es necesario ejecutar ningún comando adicional.

El asistente guiará por varias pantallas consecutivas:

- **Proveedor de IA:** permite elegir entre numerosos proveedores (Ollama local, OpenAI, Anthropic, Google, etc.). Si se selecciona Ollama, se puede escoger el modelo deseado de entre los que se descargaron previamente (por ejemplo, `mistral:7b`). Una vez funcionando, se pueden añadir más modelos y cambiar entre ellos en cualquier momento.
- **Workspace y Gateway:** se aceptan las rutas por defecto. El puerto del Gateway será el 18789.
- **Canales de comunicación:** el asistente pregunta si se desea configurar Telegram. Si se responde afirmativamente, solicitará el token del bot obtenido de @BotFather. El proceso se explica paso a paso en el propio asistente. También se puede omitir y configurarlo más adelante.
- **Herramientas de búsqueda web:** el asistente ofrece múltiples opciones (DuckDuckGo —que funciona sin API key—, Google Gemini, SearXNG, etc.). Se puede configurar más adelante si no se desea en ese momento.
- **Verificación:** el asistente comprobará que el Gateway se ha iniciado correctamente y mostrará un mensaje de confirmación.

Si en el futuro necesitas modificar la configuración (cambiar de proveedor de IA, añadir un canal, activar la búsqueda web, etc.), puedes re-ejecutar el asistente en cualquier momento con `openclaw onboard`. No borra configuraciones previas a menos que se elija explícitamente la opción Reset.

2. Tras completar el asistente, abre el *dashboard* web para confirmar que todo funciona:

```
openclaw dashboard
```

El comando `openclaw dashboard` arranca el gateway automáticamente si está detenido y abre la interfaz web en el navegador. Es la forma recomendada de iniciar el sistema cada vez que se enciende el servidor.

Solución de Problemas tras la Instalación

Si tras el asistente el bot no responde o el *dashboard* no carga:

- Verifica que Ollama está ejecutándose y los modelos están disponibles: `ollama list`.
- Comprueba el estado del Gateway: `openclaw gateway status`.
- Revisa los logs del Gateway: `openclaw logs -follow`.
- Si el asistente falló en algún paso, se puede re-ejecutar: `openclaw onboard`. No borra configuraciones previas a menos que se elija explícitamente la opción Reset.
- Si el dashboard no carga, verifica que el servidor responde: `ping 192.168.1.50`.

A.3.3. Primer Comando

1. Abre Telegram y busca el bot de Luxe Core AI (el que creaste con @BotFather).
2. Envía “Hola” o “Cómo está la casa”.
3. El bot te responderá con el estado actual de los dispositivos. Si es la primera vez, puede tardar unos segundos mientras se inicializan los modelos.
4. Prueba comandos sencillos: “Enciende la luz”, “Velocidad 3”, “Qué temperatura hace”.

A.4. Canales de Comunicación

El sistema se controla de dos formas complementarias:

- **Telegram:** es el canal principal para el uso cotidiano. Busca el bot de Luxe Core AI en Telegram y escribe cualquier mensaje en lenguaje natural. El bot entiende español coloquial, incluyendo expresiones como “velocidad 3”, “enciende la luz”, “cómo está la casa” o “buenos días”. Responde tanto a comandos directos como a preguntas abiertas.
- **Dashboard web:** interfaz de administración accesible en <http://192.168.1.50:18789>. Permite ver el estado detallado del sistema, consultar conversaciones anteriores, modificar archivos de configuración, gestionar la memoria de los agentes y acceder a ventanas de ajustes avanzados. Es el canal recomendado para tareas de configuración y diagnóstico.

A.5. Comandos Disponibles

El sistema reconoce más de 460 comandos en español, procesados en tres niveles según su complejidad:

- **Respuesta inmediata:** los comandos directos como “apaga la luz” se reconocen mediante patrones y se ejecutan en milisegundos, sin necesidad de modelos de IA.
- **Clasificación inteligente:** los comandos ambiguos o con variantes se analizan con un clasificador de IA ligero (1.5B parámetros) que los categoriza y ejecuta en aproximadamente 300–500 ms.
- **Conversación abierta:** las preguntas que requieren contexto o razonamiento se responden con Mistral 7B, un modelo conversacional que tarda entre 27 y 60 segundos en generar una respuesta.

No hace falta memorizar los comandos: basta con hablar de forma natural. Si el sistema no entiende algo, preguntará o delegará al nivel superior de procesamiento.

A.5.1. Iluminación

- “Enciende la luz” o “Luz on”
- “Apaga la luz” o “Luz off”
- “Modo noche” (atenúa la luz del ventilador)

A.5.2. Ventilador

- “Ventilador velocidad 3” o “Pon el ventilador a 3”
- “Ventilador off” o “Para el ventilador”
- “Ventilador máximo” (velocidad 5)

A.5.3. Enchufe Inteligente

- “Enciende el enchufe” o “Activa el humidificador”
- “Apaga el enchufe” o “Desactiva el humidificador”

A.5.4. Escenas Predefinidas

- “Modo noche” — apaga todo, modo noche en ventilador
- “Modo cine” — apaga luces, ajusta ventilador
- “Modo lectura” — luz encendida, ventilador suave
- “Apagar todo” — todos los dispositivos off

A.5.5. Consultas

- “Cómo está la casa” — estado de todos los dispositivos
- “Qué temperatura hace” — lectura actual del sensor
- “Qué temperatura hay fuera” — condiciones exteriores

A.5.6. Tabla Resumen de Comandos

Categoría	Ejemplos
Iluminación	“Enciende la luz”, “Luz on”, “Luz off”, “Modo noche”
Ventilador	“Velocidad 3”, “Pon el ventilador a 5”, “Ventilador off”, “Ventilador máximo”
Enchufe	“Enciende el enchufe”, “Activa el humidificador”, “Apaga el enchufe”
Escenas	“Modo noche”, “Modo cine”, “Modo lectura”, “Apagar todo”
Consultas	“Cómo está la casa”, “Qué temperatura hace”, “Qué humedad hay”

Tabla A.1: Resumen de comandos disponibles por categoría.

A.6. Notificaciones Proactivas

El sistema puede enviar mensajes sin que se los pidas. Por ejemplo:

- “Hace calor, he subido el ventilador a velocidad 4”
- “Se acerca una tormenta, he cerrado las ventanas”
- [RECUPERADO tras 2 reintentos] — el sistema se ha recuperado de un fallo temporal

Si el sistema tiene dudas (por ejemplo, si la temperatura sube un nivel y no sabe si actuar), te preguntará antes de hacer nada. Las auto-acciones solo se ejecutan sin consultar cuando hay certeza de que el usuario se beneficiará, como en zonas de temperatura extrema.

A.7. Seguridad

El sistema está diseñado con la privacidad y la seguridad como principios fundamentales. No obstante, el usuario debe seguir unas pautas básicas para mantener la seguridad del sistema:

- **Red aislada (air-gapped):** la principal medida de seguridad es mantener el router sin conexión a Internet. Ningún paquete de datos generado por los dispositivos IoT puede abandonar la red local, lo que elimina la mayoría de los vectores de ataque remotos.
- **Acceso físico al servidor:** el servidor debe estar en un lugar seguro y de acceso controlado. Cualquier persona con acceso físico al servidor podría detener los servicios, acceder a los archivos de configuración o leer las *Local Keys* de los dispositivos Tuya.
- **Contraseñas:** cambia la contraseña del usuario del servidor por una contraseña segura. No uses contraseñas por defecto. Si el sistema se accede por SSH, utiliza autenticación mediante par de llaves RSA en lugar de contraseña.
- **No exponer servicios a Internet:** no abras puertos del router hacia el servidor. El dashboard web y el acceso SSH deben permanecer accesibles solo dentro de la red local.
- **Actualizaciones:** mantén el sistema actualizado instalando periódicamente las actualizaciones de seguridad de Ubuntu (Sección [A.10.1](#)).

A.8. Solución de Problemas Comunes

A.8.1. Indicadores LED del ESP32

El ESP32 tiene un LED integrado que proporciona información visual sobre su estado:

- **LED encendido fijo:** el ESP32 está recibiendo alimentación y el firmware funciona correctamente. Es el estado normal durante la operación.
- **LED apagado:** el ESP32 no recibe alimentación o el firmware no se ha iniciado. Verifica la conexión USB y que el servidor está encendido.
- **LED parpadeante:** el ESP32 está en proceso de reinicio o comunicación. Si el parpadeo es continuo y no cesa, el firmware puede estar en un estado de error; prueba a desconectar y reconectar el USB.

A.8.2. Casos Frecuentes

A continuación se enumeran los problemas más comunes ordenados por probabilidad de ocurrencia:

- **El sistema no responde:** comprueba que el servidor está encendido (ping 192.168.1.50).
Servicios: `systemctl status model-router sensor-daemon`.
OpenClaw: `openclaw gateway status`.
- **El enchufe Tuya no responde:** comprueba que está encendido y que tiene IP en la red (debería ser 192.168.1.100). Verifica la conectividad con ping 192.168.1.100. Si no responde, desconéctalo y vuelve a enchufarlo; se reconectará automáticamente a la red.
- **El sensor no da lecturas:** el Sensor Daemon lee cada 60 segundos. Si lleva más de 2 minutos sin datos, reinicia el servicio: `sudo systemctl restart sensor-daemon`. Si el problema persiste, comprueba la batería del sensor (voltaje inferior a 2500 mV indica batería baja).
- **El dashboard web no carga:** verifica que la IP del servidor es accesible desde tu dispositivo. Prueba a ejecutar `openclaw dashboard` en el servidor, que arrancará el gateway si está detenido.
- **El ventilador no responde:** puede que el ESP32 esté desconectado o el puerto USB no se haya detectado. Verifica que aparece como dispositivo serie: `ls /dev/ttyUSB0`. Si no aparece, prueba otro cable USB o reinicia el ESP32 desconectándolo y reconectándolo.
- **Pérdida de conexión WiFi de los dispositivos:** reinicia el router desconectándolo de la corriente durante 10 segundos. Los dispositivos Tuya se reconectarán automáticamente al restablecerse la red.
- **El bot de Telegram no responde:** comprueba que el Gateway está activo con `openclaw gateway status`. Si lo está y el bot sigue sin responder, verifica que el token del bot es correcto (puede haberse regenerado desde @BotFather).
- **El sistema va lento o los modelos tardan en responder:** comprueba el uso de RAM con `free -h` en el servidor. Si supera el 90 %, reinicia el servicio `model-router` para liberar memoria. Considera reducir el número de modelos cargados simultáneamente si el problema es recurrente.
- **Los sensores dan lecturas incorrectas o erráticas:** verifica la batería del sensor. Si el voltaje es inferior a 2500 mV, sustitúyelas. También comprueba la distancia

entre el sensor y el servidor (más de 10 metros con obstáculos puede causar pérdida de paquetes BLE).

- **El ESP32 no se detecta en el sistema:** ejecuta `ls /dev/tty*` para ver los puertos serie disponibles. Si no aparece ningún `ttyUSB0` ni `ttyACM0`, prueba otro cable USB o reinicia el servidor. En algunos casos, es necesario instalar los drivers del conversor USB-serie (CP2102 o CH340, según la versión del ESP32).
- **Los comandos de voz no funcionan:** el sistema no incorpora entrada por voz en su versión actual (Capítulo 10). Todos los comandos deben escribirse por Telegram o el dashboard web.

A.9. Añadir un Dispositivo Nuevo

A.9.1. Añadir un Dispositivo Tuya

Los dispositivos Tuya necesitan conectarse a Internet durante el emparejamiento inicial para validar el registro contra la nube del fabricante. Como el sistema opera sobre una red aislada, se aplica la técnica de *Hotspot Spoofing*:

1. Con un teléfono móvil con conexión a datos, crea un punto de acceso WiFi temporal que tenga el **mismo nombre (SSID) y contraseña** que el router aislado.
2. Conecta el dispositivo Tuya a la corriente. Se conectará automáticamente al hotspot del móvil.
3. En el servidor, ejecuta `python -m tinytuya wizard` para extraer la *Local Key* del dispositivo.
4. Apaga el hotspot del móvil. El dispositivo Tuya, al encontrar el SSID original (el del router aislado), se conectará a la red local.
5. A partir de ese momento, todas las órdenes viajan directamente por la red local, sin depender de Internet.

La *Local Key* obtenida debe configurarse en el servidor mediante las variables de entorno `TUYA_DEVICE_ID` y `TUYA_LOCAL_KEY` (ver Sección A.12).

A.9.2. Añadir un Sensor BLE Xiaomi

Para añadir un sensor Xiaomi, asegúrate de que el Bluetooth del servidor está activo y que el sensor está cerca (menos de 10 metros, sin obstáculos metálicos entre medias). El servidor lo detectará automáticamente. La dirección MAC del sensor debe estar registrada en la configuración del Sensor Daemon para que comience a leerlo periódicamente.

A.10. Mantenimiento

El sistema está diseñado para funcionar sin intervención durante largos períodos. Las tareas periódicas necesarias son:

A.10.1. Actualizaciones de Software

Para actualizar el sistema, conecta el servidor a Internet temporalmente mediante la pasarela de contingencia (Capítulo 5) y ejecuta:

```
sudo apt update && sudo apt dist-upgrade
```

Tras la actualización, reinicia los servicios del sistema:

```
sudo systemctl restart model-router sensor-daemon  
openclaw dashboard
```

A.10.2. Sustitución de Pilas

Los sensores Xiaomi funcionan con dos pilas AAA. La duración estimada es de 6 meses con uso continuo. El sistema notifica por Telegram cuando el voltaje de la batería de un sensor baja de 2500 mV, indicando que es momento de cambiarlas.

A.10.3. Backup de Configuración

Los archivos de configuración del sistema se encuentran en `/etc/luxe/`. Para hacer una copia de seguridad:

```
tar -czf luxe-backup-$(date +%Y%m%d).tar.gz /etc/luxe/
```

Se recomienda realizar una copia de seguridad después de cualquier cambio en la configuración.

A.10.4. Restauración de Configuración

Para restaurar una copia de seguridad previa:

```
tar -xzf luxe-backup-20260601.tar.gz -C /  
sudo systemctl restart model-router sensor-daemon
```

Asegúrate de usar el nombre exacto del archivo de backup que desees restaurar. La restauración sobrescribirá los archivos de configuración existentes.

A.10.5. Restauración del ESP32

Si el ESP32 deja de funcionar correctamente o se desea actualizar su firmware:

1. Conecta el ESP32 al ordenador de desarrollo por USB.
2. Abre el proyecto en `firmware/esp32_fanlamp/` con PlatformIO.
3. Compila y flashea el firmware: `pio run -t upload`.
4. El ESP32 se reiniciará automáticamente con el nuevo firmware.

El código fuente completo del firmware está disponible en el repositorio del proyecto [33].

A.10.6. Diagnóstico con OpenClaw

Si el sistema presenta algún comportamiento anómalo, se puede pedir al agente de OpenClaw que lo diagnostique. Envía un mensaje como “Comprueba el estado de los servicios” o “El ventilador no responde” y el agente analizará los logs del sistema en busca de errores.

Para tareas de depuración más complejas que requieran un modelo de lenguaje más potente (como DeepSeek o GPT-4), se puede configurar un agente con un proveedor cloud en OpenClaw. Esto requiere conexión a Internet temporal; una vez realizado el diagnóstico, se puede volver a los modelos locales. El sistema base con modelos locales es suficiente para el funcionamiento diario y el diagnóstico de problemas comunes.

A.11. Preguntas Frecuentes (FAQ)

- **¿Puedo usar cualquier ordenador como servidor?** Sí, siempre que cumpla los requisitos mínimos (8 GB de RAM, Ubuntu Server). Un equipo con procesador moderno y 16 GB de RAM ofrecerá la mejor experiencia.
- **¿Qué pasa si se va la luz?** Cuando vuelva la corriente, el servidor se reiniciará y los servicios systemd (model-router, sensor-daemon) se reanudarán automáticamente. Para que OpenClaw vuelva a estar operativo, ejecuta `openclaw dashboard` en el servidor, que arrancará el gateway y abrirá la interfaz web. Los dispositivos Tuya y el router también se reconectarán por sí solos.
- **¿Puedo controlar el sistema desde fuera de mi casa?** El sistema está diseñado para funcionar exclusivamente en la red local. Para acceder desde fuera, sería necesario exponer el servicio a Internet, lo que contradice el principio de privacidad del proyecto. El canal Telegram utiliza los servidores de Telegram como intermediario, pero las decisiones de control se toman siempre en local.

- **¿Cuánta electricidad consume el sistema?** El servidor consume aproximadamente entre 30 y 60 W en reposo, dependiendo del hardware. El ESP32 consume menos de 1 W. El router típico consume entre 5 y 10 W. En conjunto, el coste eléctrico estimado es inferior a 5 € al mes.
- **¿Puedo añadir más sensores Xiaomi?** Sí. El sistema puede leer múltiples sensores LYWSD03MMC siempre que estén dentro del alcance Bluetooth del servidor. Cada sensor debe tener su dirección MAC registrada en la configuración del Sensor Daemon, en el archivo `/etc/luxe/sensors.conf`.
- **¿Cómo reinicio OpenClaw tras un apagón?** Ejecuta `openclaw dashboard` en el servidor. El comando arrancará el gateway automáticamente si está detenido y abrirá la interfaz web.
- **¿El sistema funciona sin Internet todo el tiempo?** Sí. Esa es su característica principal. Una vez instalado y configurado, el sistema no necesita acceso a Internet para ninguna de sus funciones críticas. Solo se requiere conexión a Internet para actualizaciones de software o para la configuración inicial de dispositivos Tuya mediante *Hotspot Spoofing*.
- **¿Qué pasa si quiero cambiar de router?** Solo necesitas que el nuevo router use la misma subred (192.168.1.0/24) y que asignes las mismas IPs estáticas a los dispositivos. Los servicios del servidor se adaptarán automáticamente.
- **¿Puedo usar Alexa o Google Home además de Telegram?** El sistema no incluye integración con asistentes de voz comerciales, ya que ello requeriría enviar datos a servidores externos, contradiciendo el principio de privacidad del proyecto. El control se realiza exclusivamente por Telegram y el dashboard web.

A.12. Checklist de Puesta en Marcha

Esta lista detalla los pasos necesarios para poner el sistema en funcionamiento por primera vez, desde que se abre la caja hasta que el primer comando funciona. Los tiempos son orientativos y pueden variar según la experiencia del usuario y la velocidad del hardware.

A.12.1. Fase 1: Conexión Física

1. Desembalar y colocar el servidor en su ubicación definitiva (cerca del router). (5 min)
2. Conectar el servidor al router mediante un cable Ethernet a un puerto LAN. (2 min)
3. Conectar el ESP32 al servidor mediante un cable USB-C. (1 min)

4. Enchufar el router, el servidor y los dispositivos Tuya a la corriente. (2 min)
5. Insertar las pilas en los sensores Xiaomi (dos pilas AAA cada uno). (2 min)
6. Asegurarse de que el ventilador FanLamp recibe corriente. (1 min)

A.12.2. Fase 2: Instalación de Software Base

1. Encender el router y esperar a que la red esté operativa. (2 min)
2. Encender el servidor y esperar a que arranque Ubuntu Server. (3 min)
3. Iniciar sesión en el servidor (físicamente o por SSH). (2 min)
4. Instalar Ollama (3 min):
`https://ollama.ai/install.sh (curl -fsSL ... | bash)`
5. Descargar los modelos de lenguaje (15 min):
`ollama pull qwen2.5-coder:1.5b`
`ollama pull mistral:7b-instruct-q4_K_M`
`ollama pull bge-m3`
6. Verificar que los modelos están disponibles con `ollama list`. (1 min)

A.12.3. Fase 3: Configuración de OpenClaw

1. Crear el bot de Telegram con @BotFather y obtener el token de la API. (5 min)
2. Instalar OpenClaw (3 min):
`https://openclaw.ai/install.sh (curl -fsSL ... | bash)`
3. Seguir el asistente interactivo de configuración (proveedor de IA, canales, búsqueda web). (5 min)
4. Verificar que el Gateway se ha iniciado correctamente. (1 min)

A.12.4. Fase 4: Verificación del Sistema

1. Abrir el *dashboard* web con `openclaw dashboard`. (1 min)
2. Enviar un mensaje de prueba desde Telegram o el chat web. (2 min)
3. Comprobar que el ventilador responde a los comandos. (2 min)
4. Comprobar que el enchufe Tuya responde a los comandos. (2 min)

5. Confirmar que el sensor Xiaomi muestra lecturas de temperatura y humedad.
(2 min)

Tiempo total estimado: entre 45 minutos y 1 hora, dependiendo de la velocidad de descarga de los modelos y la familiaridad con el proceso.

Manual de Código

Este anexo recoge los fragmentos de código más significativos del proyecto. No se incluye el código completo de cada archivo (eso duplicaría sin sentido el repositorio) sino los extractos que mejor ilustran las decisiones de diseño y el funcionamiento interno del sistema. El código fuente completo está disponible en el repositorio público del proyecto [33].

B.1. Enrutador de Tres Niveles (Model Router v2)

El núcleo del enrutador está en `server/model_router/router.py`. El siguiente fragmento muestra el bucle principal de clasificación:

```
async def route_message(self, message: str) -> RouterResponse:
    # Tier-0: Zero-inference --- patrón directo, sin LLM
    tier0_result = self.tier0.match(message)
    if tier0_result != Tier0Result.UNKNOWN:
        return await self._execute_tier0(tier0_result)

    # Tier-1: Clasificador 1.5B --- si está disponible
    if self.circuit_breaker.state != CircuitState.OPEN:
        try:
            tier1_result = await self._classify_tier1(message)
            if tier1_result != Classification.DELEGAR:
                return tier1_result
        except ModelUnavailableError:
            self.circuit_breaker.record_failure()

    # Tier-2: Razonador Mistral 7B --- carga bajo demanda
    return await self._reason_tier2(message)
```

Notas: La función `match()` de Tier 0 implementa 462 patrones exactos más expresiones regulares. El `circuit_breaker` evita intentar clasificar si el modelo 1.5B está caído.

B.2. Sistema de Zero-Inference (Tier 0)

El motor de *pattern matching* de Tier 0 es el nivel más utilizado del enrutador, capturando aproximadamente el 92 % de los comandos diarios. Su funcionamiento es simple pero efectivo: cada patrón se define como una tupla de (expresión regular, categoría, acción) y se evalúa secuencialmente hasta encontrar una coincidencia. El siguiente fragmento de `server/model_router/config.py` ilustra la estructura:

Fragmento representativo de los 462 patrones de Tier 0

```
PATTERNS = [
    # Iluminación (35 variantes)
    (r"enciende la luz|luz on|luces", "light", "on"),
    (r"apaga la luz|luz off|luces off", "light", "off"),
    (r"modo noche|buenas noches", "light", "night-mode"),

    # Ventilador (50 variantes)
    (r"ventilador (velocidad )?(?P<speed>[1-5])", "fan", "set-speed"),
    (r"ventilador (off|apaga|para)", "fan", "off"),
    (r"ventilador (máximo|max|a tope)", "fan", "max"),

    # Escenas (40 variantes)
    (r"modo cine|modo peli", "scene", "cinema"),
    (r"apagar todo|todo off", "scene", "all-off"),
    (r"modo estudio|modo trabajar", "scene", "study"),

    # Consultas rápidas (15 variantes)
    (r"cómo está la casa|estado", "query", "status"),
    (r"qué temperatura (hace|hay)", "query", "temperature"),
    (r"qué humedad hay", "query", "humidity"),
]

class Tier0Matcher:
    def match(self, message: str) -> Tier0Result:
        msg_lower = message.lower().strip()
        for regex, category, action in PATTERNS:
            m = re.search(regex, msg_lower)
            if m:
                params = m.groupdict()
                return Tier0Result(category, action, params)
        return Tier0Result.UNKNOWN
```

Los patrones incluyen variantes con muletillas (“eh”, “pues”), dialectalismos andaluces (“arroza”, “apaga”) y formas abreviadas (“vel3”, “luz off”). La función `match()` se ejecuta en menos de 1 ms en RAM, y aproximadamente 1.5–4 s cuando implica comandar hardware real a través del puente ESP32.

B.3. Daemon de Sensorización Continua

El `sensor_daemon.py` ejecuta un bucle infinito con una cadencia de 60 segundos. El siguiente fragmento muestra la lógica de almacenamiento atómico:

```
def _atomic_write_json(data: dict, path: Path) -> None:
    """Escribe datos JSON con atomicidad (write + rename)."""
    tmp = path.with_suffix('.tmp')
    tmp.write_text(json.dumps(data, indent=2) + '\n')
    tmp.rename(path)
```

B.4. Asesor de Confort Térmico (Comfort Advisor)

El cálculo del índice de calor NOAA y la asignación de zonas térmicas se implementa en `server/model_router/comfort.py`:

```
def evaluate_comfort(temp_c: float, humidity_pct: float) -> ComfortResult:
    temp_f = temp_c * 9.0 / 5.0 + 32.0

    # Índice de calor (Rothfusz)
    hi_f = (-42.379 + 2.04901523 * temp_f
            + 10.14333127 * humidity_pct
            - 0.22475541 * temp_f * humidity_pct
            - 6.83783e-3 * temp_f**2
            - 5.481717e-2 * humidity_pct**2
            + 1.22874e-3 * temp_f**2 * humidity_pct
            + 8.5282e-4 * temp_f * humidity_pct**2
            - 1.99e-6 * temp_f**2 * humidity_pct**2)

    hi_c = (hi_f - 32.0) * 5.0 / 9.0

    if hi_c < 23:
        zone = ThermalZone.FRESH
    elif hi_c < 26:
        zone = ThermalZone.COMFORTABLE
    elif hi_c < 28:
        zone = ThermalZone.WARM
    elif hi_c < 31:
        zone = ThermalZone.HOT
    elif hi_c < 34:
        zone = ThermalZone.VERY_HOT
    else:
        zone = ThermalZone.EXTREME

    return ComfortResult(heat_index=hi_c, zone=zone, ...)
```

B.5. Control del Ventilador FanLamp (Anuncios BLE)

El script `scripts/fanlamp_bt.py` envía comandos al ventilador mediante anuncios BLE:

```
def send_command(cmd_name: str):
    subprocess.run(['sudo', 'systemctl', 'stop', 'bluetoothd'],
                   check=True)
    group1, group2 = COMMANDS[cmd_name]
    for _ in range(5):
        subprocess.run(['sudo', 'btmgmt', 'add-adv', '-c', '-d',
                        group1, '-u', group1], check=True)
        time.sleep(0.05)
        subprocess.run(['sudo', 'btmgmt', 'add-adv', '-c', '-d',
```

```

        group2, '-u', group2], check=True)
    time.sleep(0.05)
    subprocess.run(['sudo', 'systemctl', 'start', 'bluetoothd'],
        check=True)

```

B.6. Integración Tuya (Protocolo Local 3.5)

El módulo `server/integrations/tuya_devices.py` encapsula la comunicación con el enchufe inteligente:

```

@dataclass
class TuyaDeviceConfig:
    device_id: str
    local_key: str
    address: str          # 192.168.1.100
    version: float = 3.5
    timeout: int = 5

class TuyaSmartPlug:
    def __init__(self, config: TuyaDeviceConfig):
        self._device = tinytuya.OutletDevice(
            config.device_id, config.address, config.local_key)
        self._device.set_version(config.version)

    def is_on(self) -> bool:
        status = self._device.status()
        return status.get('dps', {}).get('1', False)

    def set_state(self, on: bool) -> None:
        self._device.set_value('1', on)

    def toggle(self) -> None:
        self.set_state(not self.is_on())

```

B.7. Lectura GATT de Sensores Xiaomi

El módulo `server/integrations/ble_sensors.py` implementa la lectura de los sensores LYWSD03MMC mediante BLE GATT:

```

SERVICE_UUID = "ebe0ccb0-7a0a-4b0c-8a1a-6ff2997da3a6"
CHAR_UUID     = "ebe0ccc1"

async def read_sensor(address: str) -> SensorReading:
    async with BleakClient(address) as client:
        raw = await client.read_gatt_char(CHAR_UUID)

        temp_raw = struct.unpack_from('<h', raw, 0)[0]
        humidity = raw[2]
        battery = struct.unpack_from('<H', raw, 3)[0]

    return SensorReading(
        temperature_c=temp_raw / 100.0,
        humidity_pct=float(humidity),
        battery_mv=battery,
        rssi=client.rssi
    )

```

B.8. Auto-Recuperación (Circuit Breaker)

El *circuit breaker* se implementa en el propio router:

```
class CircuitBreaker:
    CLOSED = "CLOSED"
    OPEN = "OPEN"
    HALF_OPEN = "HALF_OPEN"

    def __init__(self, fail_threshold: int = 3,
                 recovery_timeout: int = 30):
        self.fail_count = 0
        self.state = self.CLOSED
        self.last_fail: float = 0

    def record_failure(self):
        self.fail_count += 1
        if self.fail_count >= self.fail_threshold:
            self.state = self.OPEN
            self.last_fail = time.time()

    def try_acquire(self) -> bool:
        if self.state == self.OPEN:
            if time.time() - self.last_fail > self.recovery_timeout:
                self.state = self.HALF_OPEN
                return True
            return False
        return True
```


Glosario de Acrónimos

Sigla	Significado
AC	Aire Acondicionado (<i>Air Conditioning</i>)
ADR	Registro de Decisión Arquitectónica (<i>Architecture Decision Record</i>)
AI	Inteligencia Artificial (<i>Artificial Intelligence</i>)
API	Interfaz de Programación de Aplicaciones (<i>Application Programming Interface</i>)
ASHRAE	Sociedad Americana de Ingenieros de Calefacción, Refrigeración y Aire Acondicionado (<i>American Society of Heating, Refrigerating and Air-Conditioning Engineers</i>)
BLE	Bluetooth de Baja Energía (<i>Bluetooth Low Energy</i>)
CNMC	Comisión Nacional de los Mercados y la Competencia
DHCP	Protocolo de Configuración Dinámica de Host (<i>Dynamic Host Configuration Protocol</i>)
DOI	Identificador de Objeto Digital (<i>Digital Object Identifier</i>)
DPS	Punto de Datos (<i>Data Point</i>)
DNS	Sistema de Nombres de Dominio (<i>Domain Name System</i>)
DTR	Terminal de Datos Listo (<i>Data Terminal Ready</i>)
GATT	Perfil Genérico de Atributos (<i>Generic Attribute Profile</i>)
GII	Grado en Ingeniería Informática
GPU	Unidad de Procesamiento Gráfico (<i>Graphics Processing Unit</i>)
HCI	Interfaz de Control de Host (<i>Host Controller Interface</i>)
HR	Humedad Relativa
HVAC	Calefacción, Ventilación y Aire Acondicionado (<i>Heating, Ventilation and Air Conditioning</i>)

Sigla	Significado
IDE	Entorno de Desarrollo Integrado (<i>Integrated Development Environment</i>)
IEEE	Instituto de Ingenieros Eléctricos y Electrónicos (<i>Institute of Electrical and Electronics Engineers</i>)
IoT	Internet de las Cosas (<i>Internet of Things</i>)
IP	Protocolo de Internet (<i>Internet Protocol</i>)
JSON	Notación de Objetos JavaScript (<i>JavaScript Object Notation</i>)
JSONL	<i>JSON Lines</i>
LAN	Red de Área Local (<i>Local Area Network</i>)
LIN	Red de Interconexión Local (<i>Local Interconnect Network</i>)
LLM	Modelo de Lenguaje de Gran Escala (<i>Large Language Model</i>)
LRU	Menos Usado Recientemente (<i>Least Recently Used</i>)
LTS	Soporte a Largo Plazo (<i>Long Term Support</i>)
MAC	Control de Acceso al Medio (<i>Media Access Control</i>)
ML	Aprendizaje Automático (<i>Machine Learning</i>)
MVP	Producto Mínimo Viable (<i>Minimum Viable Product</i>)
NAT	Traducción de Direcciones de Red (<i>Network Address Translation</i>)
NOAA	Administración Nacional Oceánica y Atmosférica (<i>National Oceanic and Atmospheric Administration</i>)
NWS	Servicio Meteorológico Nacional (<i>National Weather Service</i>)
RAM	Memoria de Acceso Aleatorio (<i>Random Access Memory</i>)
RSA	Rivest-Shamir-Adleman
RSSI	Indicador de Potencia de Señal Recibida (<i>Received Signal Strength Indicator</i>)
RTX	<i>Ray Tracing Texel eXtreme</i>
SIM	Módulo de Identificación de Suscriptor (<i>Subscriber Identity Module</i>)
SQL	Lenguaje de Consulta Estructurada (<i>Structured Query Language</i>)
SSH	<i>Secure Shell</i>
SSID	Identificador de Conjunto de Servicios (<i>Service Set Identifier</i>)
TFG	Trabajo de Fin de Grado
TinyML	Aprendizaje Automático en Dispositivos con Recursos Limitados (<i>Tiny Machine Learning</i>)
TTL	Tiempo de Vida (<i>Time To Live</i>)
UCO	Universidad de Córdoba
USB	Bus Serie Universal (<i>Universal Serial Bus</i>)
UUID	Identificador Universal Único (<i>Universally Unique Identifier</i>)

Sigla	Significado
VPN	Red Privada Virtual (<i>Virtual Private Network</i>)
WAN	Red de Área Extensa (<i>Wide Area Network</i>)
Wi-Fi	<i>Wireless Fidelity</i>
YAML	<i>YAML Ain't Markup Language</i>